

Multilevel Capabilities of CuMnAs Memory Cells

Ben C Potter & Alexander L Heaton
14299460 & 14303769
School of Physics and Astronomy
University of Nottingham

Monday 7th June 2021



Abstract

Research into the multilevel capabilities of antiferromagnets, such as CuMnAs, offers interesting routes for the advancement of neuromorphic computing as well as the advancement in conventional computing. This investigation can be split into two distinct objectives. The first, to produce a computational model of an antiferromagnet. This simulation was created using Python. The simulation utilised the Monte Carlo method to model the antiferromagnet, in which the program randomly samples an individual moment and calculates its energy. The model would recalculate and adjust as it attempts to reach an equilibrium. The second objective was to confirm the multilevel nature of a CuMnAs antiferromagnetic chip. This was achieved by initially finding the threshold current. The threshold current is the point where the Néel spin-orbit torque effect (a specialised form of the inverse spin galvanic effect) experienced by the physical CuMnAs sample is large enough to apply a torque that would rotate the magnetic moments parallel to the direction of the writing pulse. To find the threshold a writing current was pulsed once in a chosen direction and then another pulse was applied perpendicular to that direction, after each pulse the resistance was monitored over a set time. At the threshold, a clear and defined exponential relationship between resistance and time would be observed. The threshold current was found to lie at 18.4 mA. Upon the discovery of the threshold, the multilevel nature could be investigated. This was completed by injecting multiple writing pulses in the first direction and then an equal amount in the perpendicular direction. For confirmation of the multilevel nature, it would be expected that distinct resistance levels would be observed. This was the case for this experiment.



Contents

1	Introduction	4
1.1	Antiferromagnets	4
1.2	Chip Writing	5
1.2.1	Inverse Spin Galvanic Effect	5
1.2.2	Néel Spin-Orbit Torques	6
1.3	Chip Reading	7
1.3.1	Anisotropic Magnetoresistance	7
1.3.2	Decay Curve	9
2	Methodology	10
2.1	Computational Methodology	10
2.2	Experimental Methodology	11
2.2.1	Experimental Setup	11
2.2.2	Threshold Current	13
2.2.3	Multilevel Nature	14
3	Results	15
3.1	Computational Result	15
3.2	Experimental Results	19
3.2.1	Threshold Current	19
3.2.2	Multilevel Nature	24
4	Discussion of Results	34
4.1	Modelling	34
4.2	Experimental Discussion	35
4.2.1	Threshold Current	38
4.2.2	Multilevel	39
5	Conclusions	42
6	Appendix	46
A	Experimental Code	46
A.1	Resistance Measurement and Graphing	46
A.1.1	Resistance Measure - resistMeasure.py	46
A.1.2	Resistance Graphing - resistGraph.py	48
A.2	Pulsing and Measurement - pulseAndMeasure.py	51
A.3	Graphing	59
A.3.1	Single-level Graphing - singlelevelGraph.py	59
A.3.2	Multilevel Graphing - multilevelGraph.py	62
A.4	Analysis	66
A.4.1	Single Level Decay Analysis - decayAnalysis.py	66
A.4.2	Multilevel Gap Analysis: Pulse Current - splitPucAnalysis.py	70
A.4.3	Multilevel Gap Analysis: Pulse Separation - splitSepAnalysis.py	77



B	Modelling Code	83
B.1	Simulation - modelSimulation.py	83
B.2	Graphing - modelAnalysis.py	87
C	Notes	89



1 Introduction

1.1 Antiferromagnets

Antiferromagnetic materials possess many advantages over their ferromagnetic counterparts: they are invisible to external magnetic probing; possess ultra-fast spin dynamics (around a thousand times faster than a ferromagnet); and a wide range of materials are antiferromagnetic at room temperature from insulators to superconductors and everything in-between. This is not the case for ferromagnetic materials which are mostly metals [1][2][3].

The defining property of an antiferromagnetic material is its zero-net magnetisation. For the case of CuMnAs, the subject of this paper, the zero-net magnetisation takes the form of an antiparallel alignment of its coupled Mn moments. When a material has no net magnetisation there will be no stray field interaction. Without the presence of stray fields the antiferromagnet will not interact with any neighbouring materials [4]. Such a property allows for antiferromagnet memory to have the capacity for higher densities over ferromagnetic bits, which produce stray fields, thus interacting with neighbouring bits [5]. It is this interaction that makes ferromagnetic memories susceptible to corruption from magnetic fields, an issue not faced by antiferromagnetic materials [6]. In addition, CuMnAs offers many more advantageous properties when compared to other antiferromagnets. It possesses strong magnetotransport effects and the ability to be easily integrated into established semiconductor technologies due to the materials' growth process [7][8]. Combine the denser packing and the possible increase in switching speed, which is achievable through the ultra-fast spin dynamics, and this form of memory could be a disruptive technology [9][10].

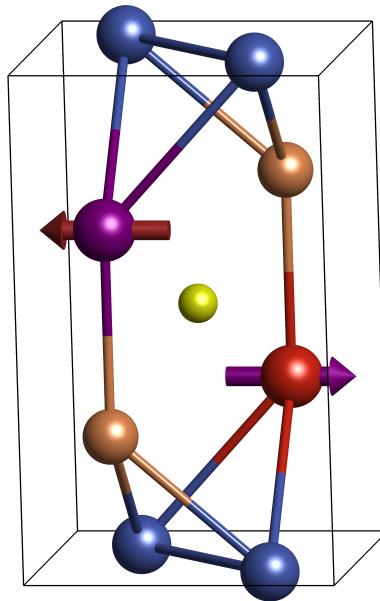


Figure 1: This image shows the sublattice structure of CuMnAs. The sublattice is comprised of one Mn-A atom (shown in red) coupled with one Mn-B atom (shown in purple). The yellow ball represents the centre of inversion within the sublattice. This image was produced by Jakub Zelezny.

Research published by Wadley *et al.* [11] in 2013 announced the candidacy of the antiferromagnet CuMnAs for use in antiferromagnetic spintronics. CuMnAs is a centrosymmetrical crystal that can be separated into two inversion asymmetrical halves [12]. This breakthrough was important for antiferromagnetic spintronics because of the crystal's unique symmetry, a symmetry that is instrumental in controlling the antiferromagnet efficiently [13]. In CuMnAs the two moments present in the inversion halves are coupled in an antiparallel configuration, hence the material possesses a zero-net magnetisation. However, due to the sublattice's inversion asymmetry, it is possible for an electric field to generate a lattice site dependant spin polarisation texture and resultant torque, this is called Néel spin-orbit torque and was a crucial breakthrough for the field of relativistic antiferromagnet spintronics. Néel spin-orbit torque can be used to manipulate the magnetic ordering of antiferromagnets and investigate CuMnAs' multilevel capabilities and will be discussed further in section 1.2 [14].

1.2 Chip Writing

The electrical writing process used in this investigation utilises Néel spin-orbit torque, an effect that can only be generated in crystals with symmetry as discussed above. Passing a current through such a crystal leads to the generation of an alternating internal magnetic field that couples strongly to the magnetic ordering allowing the rotation of the Néel axis [15]. Utilising two directions of current flow, the current channels positioned perpendicular to each other (bisecting at the chip's centre), allows for control of the magnetic moments through a 90-degree rotation. However, to understand the origins of Néel spin-orbit torque other effects need to be described.

1.2.1 Inverse Spin Galvanic Effect

The inverse spin galvanic effect (ISGE) or Edelstein effect is described as the imbalanced (non-equilibrium) generation of spin polarisation in a perpendicular direction to an applied electrical field external to the system due to inversion asymmetry, the field can also be equated to the injection of a current. The effect is the result of Rashba spin-orbit coupling which aligns spins dependent upon their momentum, leading to the polarization of an injected current, achievable with or without the presence of magnetic ordering [16].

It is a widely accepted belief in the scientific community that any experimentation on centrosymmetrical antiferromagnetic materials, in equilibrium, would yield no net polarization of a current when passed through a sample [17][18]. This is also the case for centrosymmetrical antiferromagnets under an applied field.

Whilst it is the case that no net polarization occurs through a centrosymmetrical crystal, the belief that no polarization occurs within a centrosymmetrical crystal's unit cell was first opposed by a postulation made by Xie *et al.* [19] in 2014. In this paper researchers presented a theory: spin texture could occur within a Bi_2Se_3 crystal unit cell under equilibrium conditions. Independently and simultaneously another paper was written by Železný *et al.* [12]. In this paper, it was hypothesised that the sub-unit cell effect could be used in conjunction with an applied field to produce a sub-unit cell field.

Železný *et al.* [12] set forth the proposal that the site inversion asymmetry in a centrosymmetrical crystal could lead to a local Rashba effect (on an atomic scale), the paper even outlined the device geometry needed to produce the effect. The resultant torque on the antiferromagnetic ordering parameter was coined by Železný *et al.* [12] as the Néel spin-orbit



torque. For the case of CuMnAs each sublattice would generate spin polarisation around the Mn-A and Mn-B moments, these would be equal in magnitude but would also have an opposite sign. The opposite polarisation sign leads to a coherent torque because of the opposing directions of the two magnetic moments.

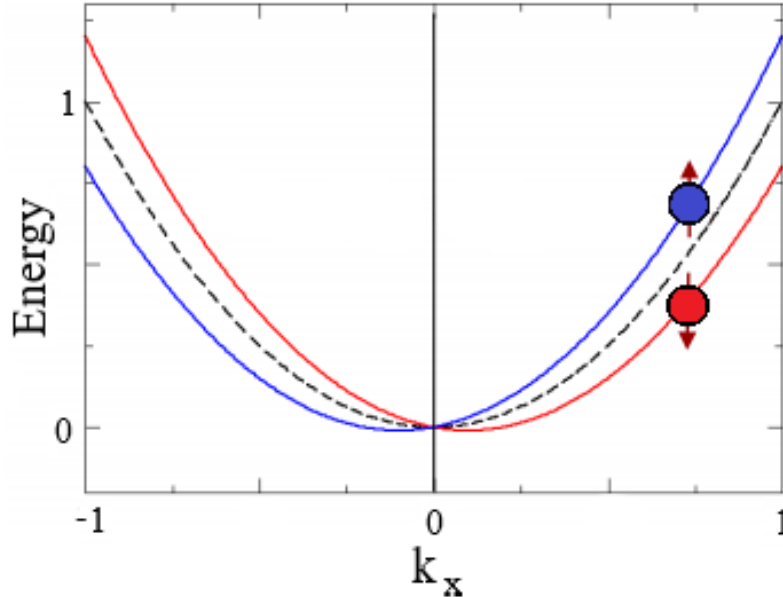


Figure 2: This image shows the simplified energy distribution of the charge carriers in CuMnAs crystal. In the black, is the net distribution which shows no net polarization of the current. In blue is the polarisation of the current as it passes Mn-A and in red is the polarisation of the current as it passes Mn-B.

1.2.2 Néel Spin-Orbit Torques

The Néel spin-orbit torque (NSOT) phenomenon results in the reorientation of magnetic moments under an alternating effective field, it can also be described as the rotation of the antiferromagnetic Néel axis. This type of spin-orbit torque is only achievable in an antiferromagnetic material with two sublattices with inversion asymmetry. The effective fields that result in NSOT are affected by both the direction of the input current and the crystal's symmetry [19]. The effective field that each moment experiences is derived from the two mirrored inversion partner sublattices that constitute the centrosymmetric CuMnAs unit cell. When a current is run through the two sublattices, the current will obtain an opposite non-equilibrium polarization, as described in section 1.2.1. The magnetic moments (that are in equilibrium) exchange-couple with the induced spin polarization to generate a torque [20].

The rate of change of the magnetisation of a particle due to an external field is described by the Landau-Lishfitz-Gilbert equation, equation 1, the first term is the Landau-Lishfitz driving term, the second term is the Gilbert damping term, and the final term indicates that a field-like torque will be exerted on the magnetisation when net steady state polarisation is not parallel to the magnetisation. The equation is as follows:

$$\begin{aligned} \frac{d\mathbf{M}(\mathbf{r}, t)}{dt} = & -\gamma\mathbf{M}(\mathbf{r}, t) \times \mathbf{H}_{eff}(\mathbf{r}, t) + \alpha\mathbf{M}(\mathbf{r}, t) \times \frac{d\mathbf{M}(\mathbf{r}, t)}{dt} \\ & + \frac{-J_{acc}N_{Mn}S_{Mn}}{g\mu_B}\mathbf{M}(\mathbf{r}, t) \times \delta\mathbf{S} \end{aligned} \quad (1)$$

where $\mathbf{M}(\mathbf{r}, t)$ is the magnetisation field, $\mathbf{H}_{eff}(\mathbf{r}, t)$ is the effective field which is the sum of the applied anisotropy and demagnetising fields alongside the exchange interactions, α is the Gilbert damping coefficient, γ is the gyromagnetic ratio, μ_B is the Bohr magneton, g is electron g-factor (≈ 2), J_{acc} is the antiferromagnetic coupling constant, N_{Mn} is the concentration of Mn atoms, S_{Mn} is the spin of the Mn atoms, and $\delta\mathbf{S}$ is the non equilibrium component of the carrier spin density [15].

The Landau-Lishfitz-Gilbert equation can be reduced to provide the equation for the field-like torque experienced by the Mn-A and Mn-B moments. First, the torque on the Mn-A moment is simplified to

$$\vec{T}_A \sim \vec{M}_A \times \vec{P}_{A,current} \quad (2)$$

and secondly the torque on the Mn-B moment is simplified to

$$\vec{T}_B \sim \vec{M}_B \times \vec{P}_{B,current} \quad (3)$$

In these equations, $\vec{T}$ is the field-like torque that is experienced, $\vec{M}$ is the magnetisation, and $\vec{P}$ is the polarisation of the current around each Mn moment [13].

For a torque to cause a rotation within an antiferromagnetic material, each coupled moment will need to experience an opposite effective field, this would keep the moments antiparallel, and is achievable through the difference in polarisation around Mn-A and Mn-B produced by the ISGE. Otherwise, the effective field would promote a rotation of each moment in the same direction. If the effective field were to promote such a rotation, no movement of the Néel axis would occur because NSOT is far too weak to break the coupling between the Mn moments [].

1.3 Chip Reading

The bit reading process used during this investigation utilises the measurement of anisotropic magnetoresistance, a process where the resistance is dependent upon the magnetic moment's orientation compared to the direction of current injection or the magnetic moment's orientation compared to the crystalline lattice [21].

1.3.1 Anisotropic Magnetoresistance

AMR was first observed in the 1850s, it is the dependence of a material's electrical resistance due to the relative orientation of its magnetic moment axis [22]. The phenomenon arises from the actions of spin-orbit coupling and magnetisation, both are material dependant. It is observed that for most materials the AMR is at its maximum when the direction of the magnetic moment axis is parallel to the direction of the current input, this is the case as there is an increased s-d scattering probability for this orientation [15].

There are two forms of AMR measurement. R_{xx} which is the longitudinal resistance and R_{xy} which is the transverse resistance. The values for both R_{xx} and R_{xy} are found by passing



a current through a sample, the current would be run laterally and in-plane with the sample, and both R_{xx} and R_{xy} can be used during the same investigation [15]. Through the course of this paper, the current will be referred to as a reading pulse. It is interesting to note that the transverse resistance can become quite complex if the magnetisation has an out-of-plane component, this component will cause the R_{xy} to increase due to the contribution of the anomalous Hall effect [23]. The value for R_{xx} is found using equation 4 and the value for R_{xy} is found using equation 6 [24].

$$R_{xx} = \frac{\Delta\rho_{xx}}{\rho_{average}} - \left(\frac{\Delta\rho_{xx}}{\rho_{average}} \right)_{min} \quad (4)$$

Where R_{xx} is the longitudinal resistance, $\rho_{average}$ is the average value of resistivity over a 2π rotation, ρ_{xx} is longitudinal resistivity, and $\left(\frac{\Delta\rho_{xx}}{\rho_{average}} \right)_{min}$ is the minimum value for $\frac{\Delta\rho_{xx}}{\rho_{average}}$.

$$\begin{aligned} \frac{\Delta\rho_{xx}}{\rho_{average}} = & C_{NC}\cos(2\theta) + C_{UC}\cos(2\theta) + C_{CC}\cos(4\phi) \\ & + C_{NC,CC}\cos(4\phi - 2\theta) \end{aligned} \quad (5)$$

Where C_{NC} is the non-crystalline coefficient, C_{UC} is the uni-axial crystalline coefficient, C_{CC} is the cubic crystalline coefficient, $C_{NC,CC}$ is the crossed non-crystalline and cubic crystalline coefficient, θ is the angle between reading pulse and the magnetic moment axis, and ϕ is the angle between the crystalline axis and the magnetic moment axis.

$$R_{xy} = \frac{\rho_{xy}}{\rho_{average}} - \left(\frac{\rho_{xy}}{\rho_{average}} \right)_{min} \quad (6)$$

Where R_{xy} is the transverse resistance, ρ_{xy} is the transverse resistivity, $\rho_{average}$ is the average value of resistivity over a 2π rotation, and $\left(\frac{\rho_{xy}}{\rho_{average}} \right)_{min}$ is the minimum value for $\frac{\rho_{xy}}{\rho_{average}}$.

$$\frac{\rho_{xy}}{\rho_{average}} = C_{NC}\sin(2\theta) + C_{NC,CC}\sin(4\phi - 2\theta) \quad (7)$$

AMR can be reduced to two key categories: non-crystalline AMR and crystalline AMR [25]. The first describes a material whose AMR is a function of the direction of the reading pulse and the magnetic moment axis (majority of the function), with a R_{xx} component dependant on $\cos 2\theta$. The second describes the case where a material's AMR is a function of the crystalline axis angle to the magnetic moment axis, with a R_{xx} component dependant on $\cos 2\phi$ or $\cos 4\phi$ (affected by crystalline structure and symmetries) [24]. In the case of non-crystalline AMR, materials can be split into two types. For a material with a larger resistance parallel to the reading pulse, it is said to have "positive AMR". For a material with a larger resistance perpendicular to the reading pulse, it is said to have "negative AMR". However, for crystalline AMR material these terms are unimportant.

1.3.2 Decay Curve

The anisotropic magnetoresistance will decay after time. In a paper by Kašpar *et al.* [26], which explores resistance decay seen in thin-film antiferromagnetic materials, they show that the decay curve immediately following magnetic moment switching can be described by a Kohlrausch stretched exponential function. The paper used the equation,

$$f_{\beta}(t) \sim \exp \left[- \left(\frac{t}{\tau_1} \right)^{\beta} \right] \quad (8)$$

where τ_1 is the relaxation time, t is time, and β is a stretching exponent base around the dimensionality of the diffusion equation that describes the relaxation process.

The relaxation time is temperature dependent, as shown by,

$$\tau_1 = \tau_0 \exp \frac{E_1}{k_B T} \quad (9)$$

where τ_1 is the relaxation time, τ_0 is the attempt time, E_0 is the activation energy, and T is the temperature. This equation was based on the findings of Phillips [27], whose paper explored the theories of ideal stretched exponential relaxation.



2 Methodology

2.1 Computational Methodology

The aim for the computational portion of this investigation was to produce a simulation of an antiferromagnet in equilibrium conditions with no external stimuli. The simulation was created in the open-source programming language of Python and utilises the Monte Carlo method. This method is a broad class of computational algorithm that depends on random sampling to produce a probabilistic model used to approximate a numerical solution.

For this simulation, an array of 100x100 cells was modelled, each cell containing a magnetic moment in a random orientation. A pair of cells would constitute a coupled antiferromagnetic pair and would remain in an antiparallel alignment for the simulation. Initially, the energy of the moment needed to be modelled. Every moment's energy was calculated as the sum of the anisotropic energy (dependent on the moments orientation against the easy axis) and the exchange energy (dependent on the moments orientation against its four closest neighbours), these calculations can be seen in Appendix B.1.

Before these calculations took place the constants and parameters were changed for the ease of the user. The Boltzmann constant was set to 1 and hence adjusted the exchange energy scale by a factor of 100 and the anisotropic energy scale by a factor of 2.

The energy of each moment provides information on the stability of its orientation, as the moment will always attempt to move into the lowest possible energy state. To simulate the reorientation of the moments, a random cell would be selected by the programme. Within a loop the energy of the moment would be calculated. It would then be rotated by a small random amount before the energy was recalculated. If the second state had a lower energy than the first it would result in a Boltzmann factor larger than 1. In the case where the Boltzmann factor is larger than one, a reorientation to the second state would occur because this is energetically favourable. If the second state had a larger energy than the first state a loop would be run to see if the Boltzmann factor was larger than a random number, between 0 and 1, produced by a Maxwell-Boltzmann probability distribution. If the Boltzmann factor were larger than the random number then the moment would reorientate despite not being energetically favourable. If the Boltzmann factor was smaller than the random probability, then the moment would remain in its first state.

For this investigation an attempt to reorientate was made 10,000 times a frame and in every simulation there were 15,000 frames. The data produced by this simulation was combined to form an animation consisting of a contour plot/ heat map of the moments' angular difference, with an antiparallel alignment appearing cool and a parallel alignment appearing hot. This allowed the tracking of domain wall formation and evolution within the sample.

With the ground level in place for the simulation it was advanced so that it was no longer a fixed wall system, a new looped wall system was introduced. This meant that instead of a having moments fixed around the edge of the array, the moments on the far-left wall would interact with the moments on the far-right wall and vice versa. This change gives a simulation of an infinite sheet of antiferromagnetic material.



2.2 Experimental Methodology

2.2.1 Experimental Setup

The 60 nm thick antiferromagnetic chip used in this investigation was built using a CuMnAs circle of material. The chip has eight $10\ \mu\text{m}$ wide CuMnAs conductive lines that pass into the square centre, these were positioned as a Hall cross. The conductive arms emanate from the centre, each at an angle of 45 degrees from its nearest neighbour, between the arms the CuMnAs was etched away so that the GaAs under-layer acts as an insulator between the lines. For this investigation the lines lying along the $[1\ 0\ 0]$ and $[0\ 1\ 0]$ channels were used solely for reading pulses and the line lying along $[1\ 1\ 0]$ and $[1\ -1\ 0]$ were used solely for writing pulses. Below is a simplified schematic of the CuMnAs chip.



Figure 3: This illustration is a simplified schematic of the CuMnAs chip adjusted from an article by Wadley *et al.* [28]. The yellow lines leading to the centre are the conductive lines, the dark areas are insulators separating the conductive lines, and the dark brown circle in the middle is the antiferromagnetic material.

The chip is surrounded by a ring of twelve conductive pads. Only eight of these pads were used during this investigation, leaving four unused. The geometry and coding of the pads is shown in figure 4, the coding of each pad is used a shorthand for the Miller index. The eight pads encircling the chip are connected to a corresponding conduction plate on the surface

of the chip via a pin, these light grey conductive plates can be seen in fig 5. The pins are connected to the conductive plate in a position as close to the conductive lines. The reason for the pins position close to the centre is two fold. First, it limits the resistance readings from the chip as a whole and dials in on the resistance of the CuMnAs. Secondly, by reducing the resistance it limits the Joule heating effects that could lead to a damaged chip.

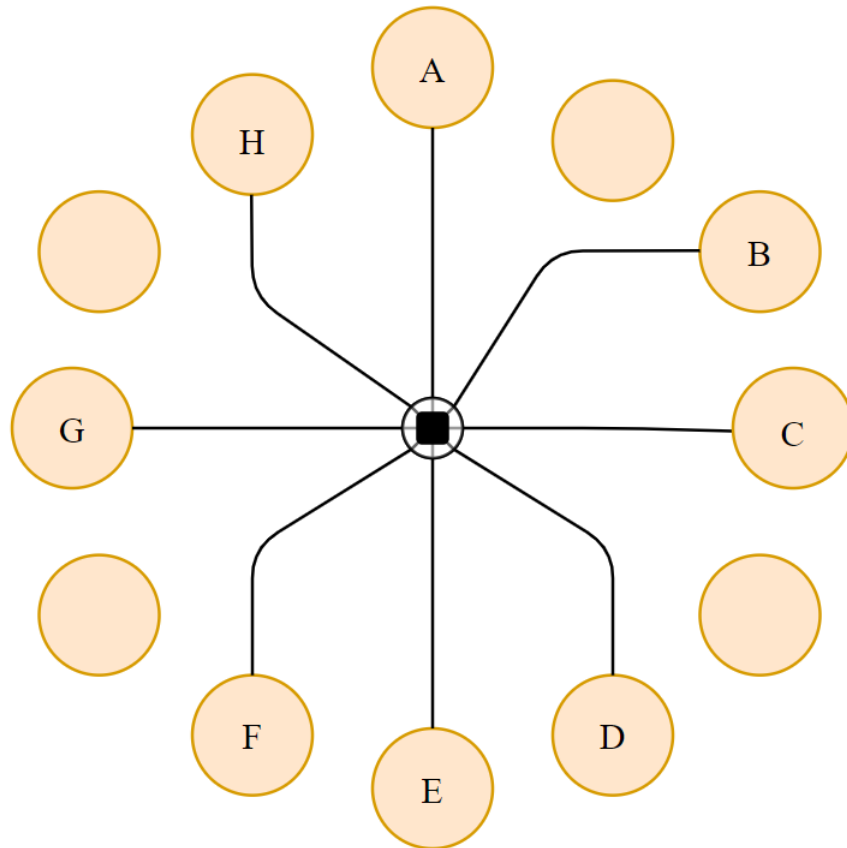


Figure 4: This image shows the simplified geometry of the connections to the antiferromagnetic chip. Each circle surrounding the chip is a conductive cylinder/pad, connected to a specific input channel. Every connection is denoted by a letter, if there is no letter on the circle it is not connected to the system.

Before the investigation could begin the chip needed to be examined. This took the form of testing the resistance of the chip through every pin, allowing us to detect any disconnected pins or issues with the chip. The resistance readings were taken via two methods, a 4-wire measurement, and a 2-wire measurement. When a 4-wire measurement was used the current was sourced from a K2461 multimeter and the resistance across the chip was recorded by the same device through a separate connection. For the case of a 2-wire measurement, the current was output by a K2461 multimeter but the resistance across the chip was recorded through a K2000 multimeter. The resistance readings were checked by plotting an I-V graph. With the low current used no switching is likely to take place, so an Ohmic response would be expected for the chip. This was confirmed in this investigation as both 2-wire and 4-wire readings produced indistinguishable results.

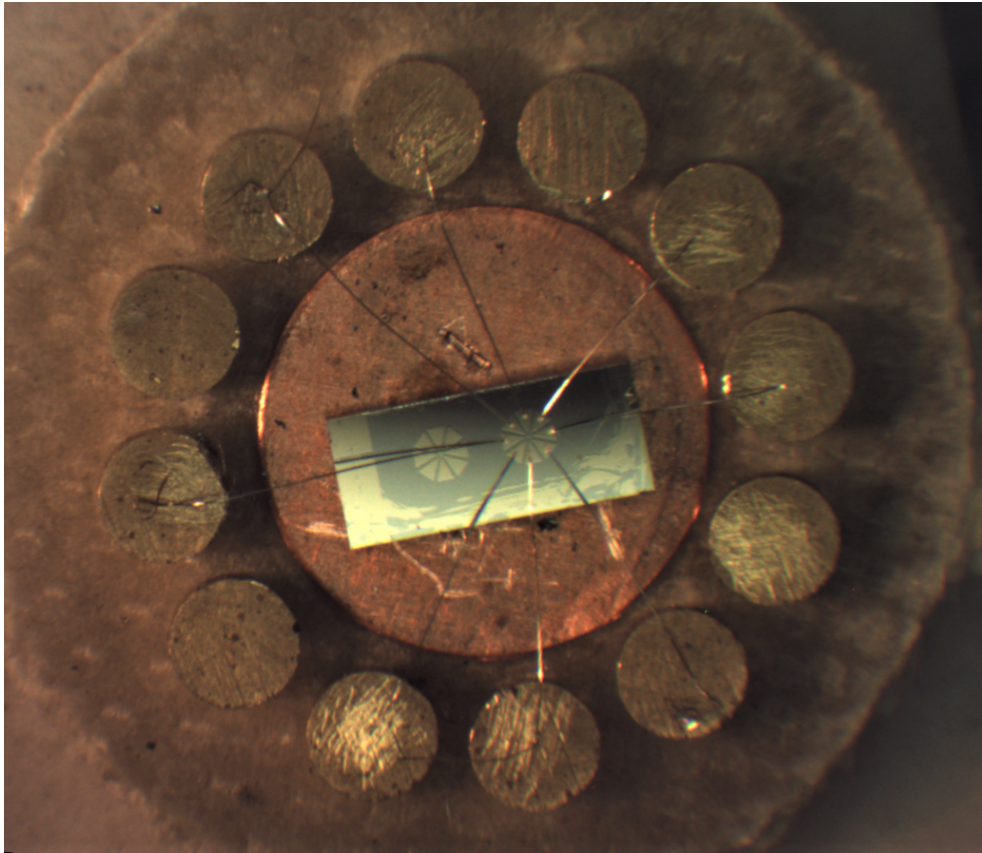


Figure 5: This image shows the chip used in this experiment. The large ring of conductive pads is seen around the edge with the pins leading to the CuMnAs centre.

2.2.2 Threshold Current

The experimental investigation's first aim was to establish the threshold current that caused switching within the chip. This was achieved with the use of a switch box connected to a multimeter in the 4-wire configuration. A code was written to inject a 1ms writing pulse before a reading pulse was taken every second in the 300 s cycle. In the early stages of this investigation two cycles were run, the first with writing pulses from $H \rightarrow D$ and the second from $B \rightarrow F$. The reading pulse was consistently fired across from $G \rightarrow C$. The experiment started with writing pulses with an amplitude of 1 mA, if no switching was evident in the R_{xx} and R_{xy} resistances then the amplitude of the current was increased by 0.5 mA.

When switching became evident in the resistance readings the increase in writing current was lowered to 0.2 mA, an additional two cycles were added, so that better human monitoring could be enabled. This was continued until a sharp increase then plateau was observed in the resistance readings per cycle. Once this was completed, the current which was found to be the threshold was used to produce a 10 cycle graph, the purpose of this was to check for any chip deformation at the threshold current. If there was deformation then the threshold current used in the experiment would be lowered.

2.2.3 Multilevel Nature

The experimental investigation's second and core aim was to gather data on the multilevel capabilities of the chip. The procedure began by injecting a series of five writing pulses through the device at a current lower than the threshold, approximately 1 mA below. For each test two cycles would be completed, a cycle consists of five pulses along $H \rightarrow D$ and then five pulses along $B \rightarrow F$. The separation time between writing pulses was limited to 2 s. This was enforced because shorter separation times have been shown to display more rapidly observable multilevel effects than their longer separation time counterparts [29]. This initial batch of testing and pulses acts as a sample to check that there is multilevel switching within the device.

Once the multilevel nature the CuMnAs was demonstrated two of the variables were adjusted so that their effects on the multilevel steps could be observed. The first of the two variables was the separation time, the magnitude of the current kept constant for this. The separation time between pulses was varied from the maximum time of 13.88 s to the minimum separation of 2.33 s, the number of pulses was kept consistent with only five writing pulse for each pulse direction. The results from these tests were combined into a singular graph to more effectively display the difference that separation time has on the multilevel response.

The second variable adjusted was the magnitude of the writing pulses. For this portion of investigation the separation time was kept at its minimum from the previous test. The writing pulses started at a magnitude 1mA below the threshold and was increased by 0.1 mA until it completed its last batch of tests at 1 mA above the threshold. The results from these tests were combined into a singular graph to more effectively display the difference that the magnitude the current has on the multilevel response.



3 Results

3.1 Computational Result

The following is a series of images taken from points within an animation created by a code to simulate an antiferromagnet. The colours in these figures correspond as so - yellow indicates that that spin is aligned along the x-axis, and blue indicates that that spin is aligned along the y-axis. The areas where the colour is between the two, where the two colours meet, is a domain wall.

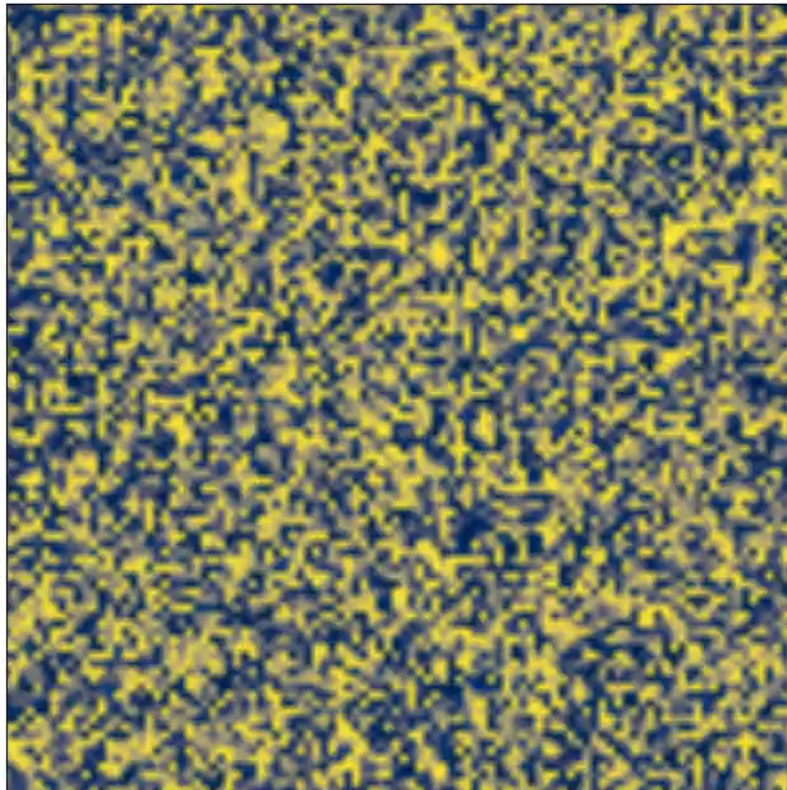
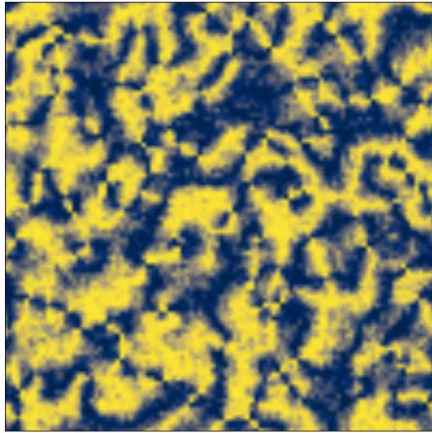
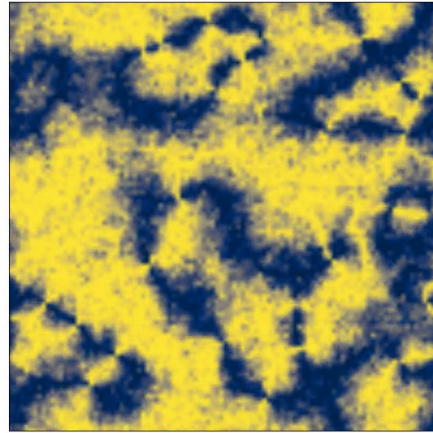


Figure 6: This image shows the starting state (0 s in) of the antiferromagnetic simulation.

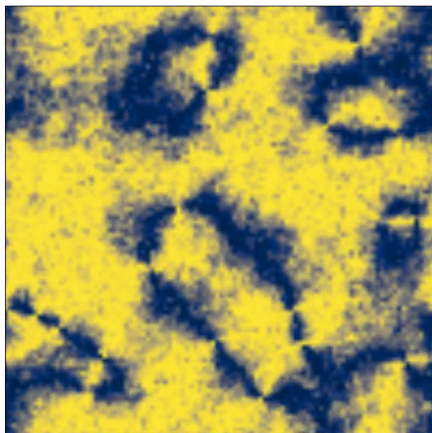
Figure 6 is the first frame from the simulation of the antiferromagnetic. The random orientation of the moments is displayed through the variegated array of colours present in the image. This provides evidence that there is no ordering before the interactions occur.



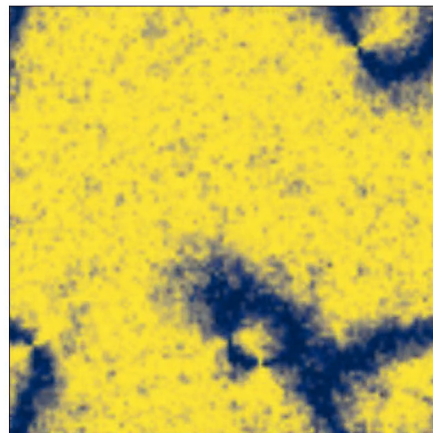
(a) 1 s into the simulation



(b) 6 s into the simulation



(c) 15 s into the simulation



(d) 30 s into the simulation

Figure 7: A collection of snapshots from the animation. The animation lasted for 83 s, each of the snapshots was taken at a time that best displays the evolution of the system.

Figure 7 is a selection of snapshots from the simulation of the antiferromagnetic material. Only 1 s in, the antiferromagnetic ordering has taken strong effect, having gone from the state in figure 6 to figure 7a. From the state in figure 7a to figure 7b, there are fewer domain walls and larger domains. This collection of domain walls is formed by the sixth second of the simulation, overlapping with one another. Going from figure 7b to figure 7c there is further dissipation of domain walls. The progression through the snapshots makes it evident that the system is attempting to evolve into a more ordered state. From figure 7c to figure 7d a much bigger domain is formed. The overall progression shows that the walls dissipate, migrate and twist as the system is attempting to reach its equilibrium state. In figures 7c and 7d a drastic increase in order can be observed, with image 7d only having a single domain wall that loops



in the corners to form a singular circle.

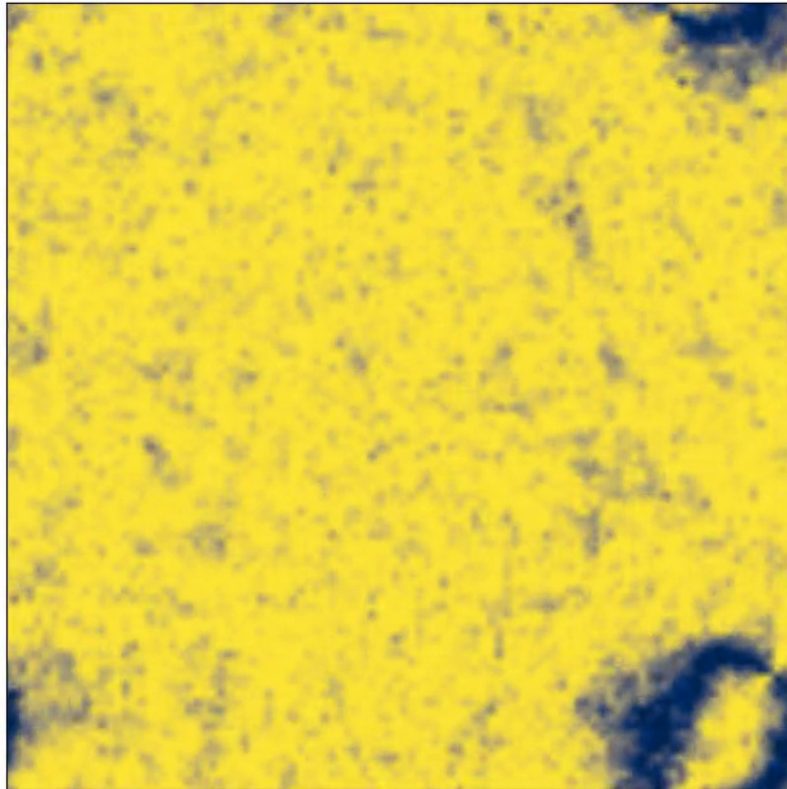


Figure 8: This image shows the final frame of the antiferromagnetic material simulation.

Figure 8 is the final frame from the simulation of the antiferromagnetic material. In the image the system is near equilibrium, this is noticeable due to the limited number of changes that have occurred between the last snapshot in figure 7d and the figure 8. This shows that the model is capable of antiferromagnetic ordering over a looped sample and hence over an infinite distance. Throughout the entire animation, when the antiferromagnetic material is less ordered, the simulation changes more. When there are larger consistent domains, there is less change.

This can be seen by plotting the mean value of $\cos(2 \cdot \theta)$ across the entire grid of spins. This gives a good estimate of just how fast the grid changes.

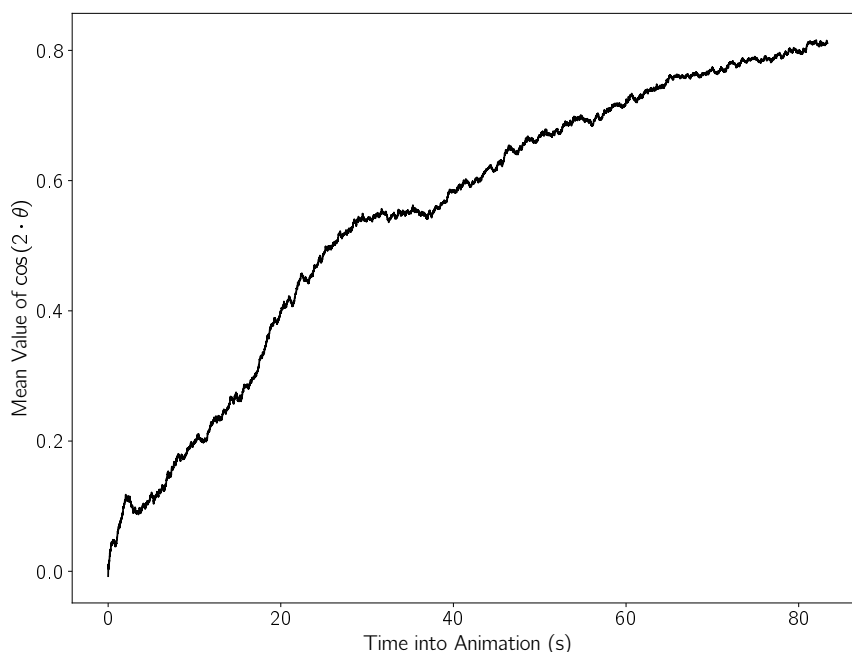


Figure 9: Graph showing the mean value of $\cos(2 \cdot \theta)$ across the entire grid of spins against the time into the video.

Figure 9 shows that the simulation does change rapidly in the beginning, slowing down over the middle, and eventually slowing down into an equilibrium state. This is logical, as the exchange mechanism uses comparisons with neighbouring spins, which means that when they're different they will cause a change, but when they're similar, they aren't affected. Also, all the angles of the spins line up along the x-axis. This is as expected as the anisotropic energy built into this model prefers this direction, further verifying the simulation.

The full animation that resulted from the simulation is available both through this link https://youtu.be/IuL_Qu0Na8g¹ and in the electronic appendices.

¹Extended Edition: <https://youtu.be/NCNCdXgqpuE>



3.2 Experimental Results

3.2.1 Threshold Current

The following set of figures were produced in the attempt to find the threshold current and decay time of the CuMnAs sample.

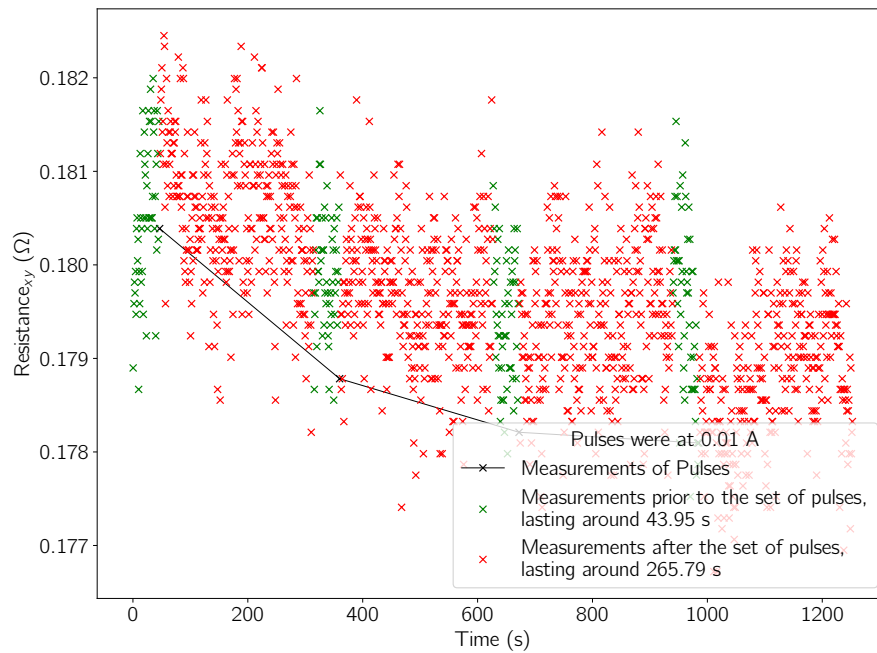


Figure 10: A graph showing the data collected during switching with a pulse current of 10 mA.

In figure 10, there is no switching occurring as the pulse current of 10 mA is not close enough to the threshold current to have any effect. The graph shows no switching behaviour whatsoever.

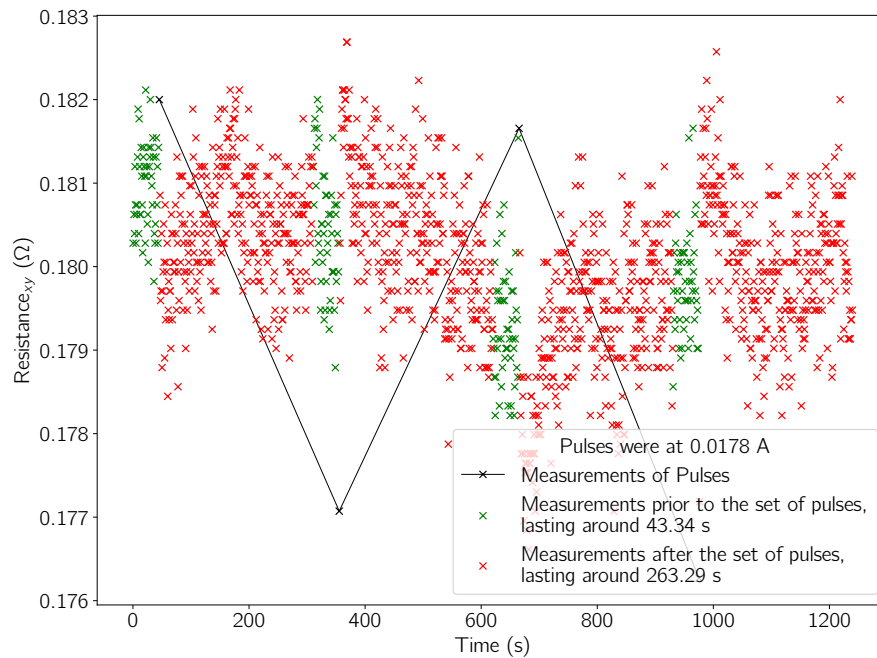


Figure 11: A graph showing the data collected during switching with a pulse current of 17.8 mA.

In figure 11 switching behaviour starts to appear. The pulse current of 17.8 mA is just enough to start to see some switching behaviour. Although it is not explicitly clear, the decay curve is starting to appear.

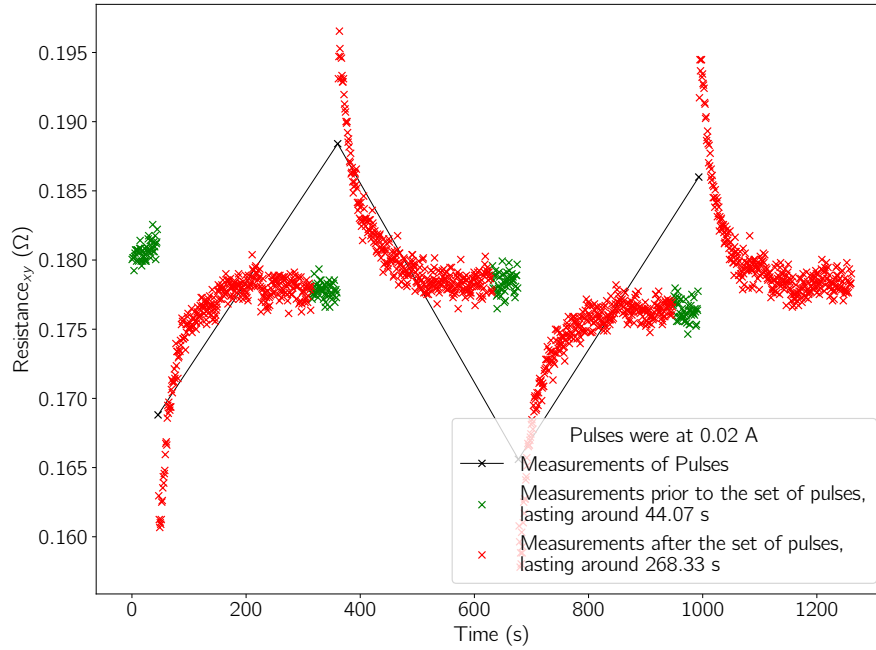


Figure 12: A graph showing the data collected during switching with a pulse current of 20 mA.

Figure 12 shows clear switching behaviour. The pulse current of 20 mA is enough to clearly demonstrate the switching behaviour of the antiferromagnetic material. The decay curve from the switching is very clear.

An overlay of decay curves can be fit to the graphs using equation 8 which later evolved to equation 10. To achieve this a scale factor was placed at the front of the equation and an offset was placed at the end in order to align the starting position away from zero.

$$R_{xy} = a \cdot \exp \left[- \left(\frac{t}{b} \right)^{0.6} \right] + c \quad (10)$$

where a was the scale factor, b represents τ_1 from equation 8, c was the offset, and in a paper written by Kašpar *et al.* [26] they found β to be 0.6. The run at 20 mA resulted in four good fits, providing evidence that the parameters were within an acceptable range. An example of the fit and results can be found in figure 13.

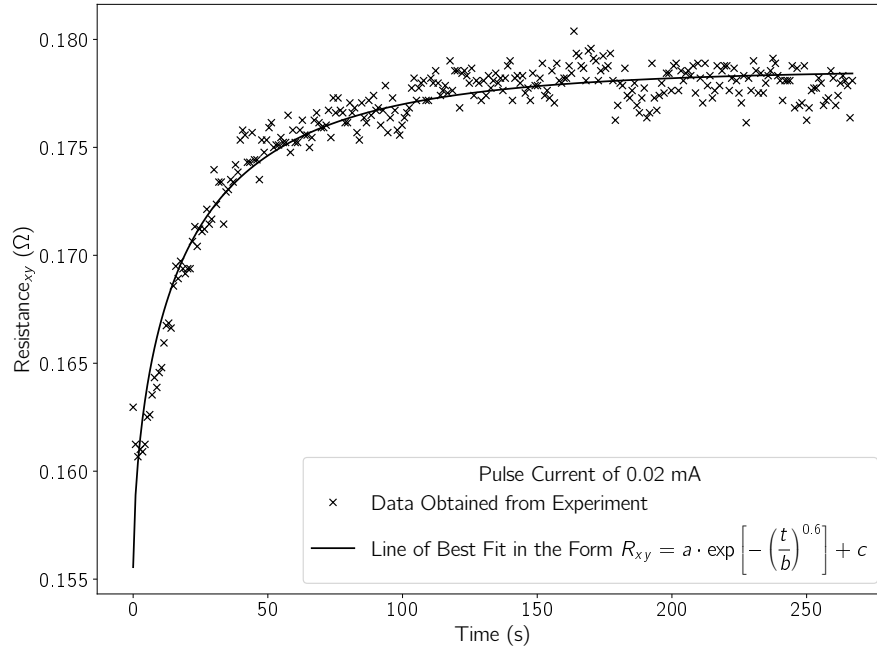


Figure 13: This shows the line of best fit corresponding to the data from immediately after switching, with R_{xy} being transverse resistance and t being time.

Loop	a (Ω)	b (s)	c (Ω)
First Negative	$-2.31 \pm 0.05 \times 10^{-2}$	$1.9 \pm 0.1 \times 10^1$	$1.786 \pm 0.001 \times 10^{-1}$
First Positive	$2.22 \pm 0.04 \times 10^{-2}$	$2.8 \pm 0.1 \times 10^1$	$1.775 \pm 0.001 \times 10^{-1}$
Second Negative	$-2.34 \pm 0.04 \times 10^{-2}$	$2.4 \pm 0.1 \times 10^1$	$1.772 \pm 0.001 \times 10^{-1}$
Second Positive	$2.17 \pm 0.05 \times 10^{-2}$	$2.1 \pm 0.1 \times 10^1$	$1.777 \pm 0.001 \times 10^{-1}$

Table 1: The values obtained from the curve fit of the pulses at 20 mA for the parameters of the equation.

The scale factor and offset can be set aside for the calculation of the relaxation time, the values obtained for b were used to find τ_1 . Combining the values and their errors gives a final value of 22.5 ± 0.3 s for τ_1 . Using these fits, will not only determine the parameters of the equation but can also be used to observe where the threshold current becomes stable.

A “fitness value” can be found for each pulse current which, when plotted, can then be used to determine the threshold current. The method for finding this “fitness value” can be found in section 4.2.1.

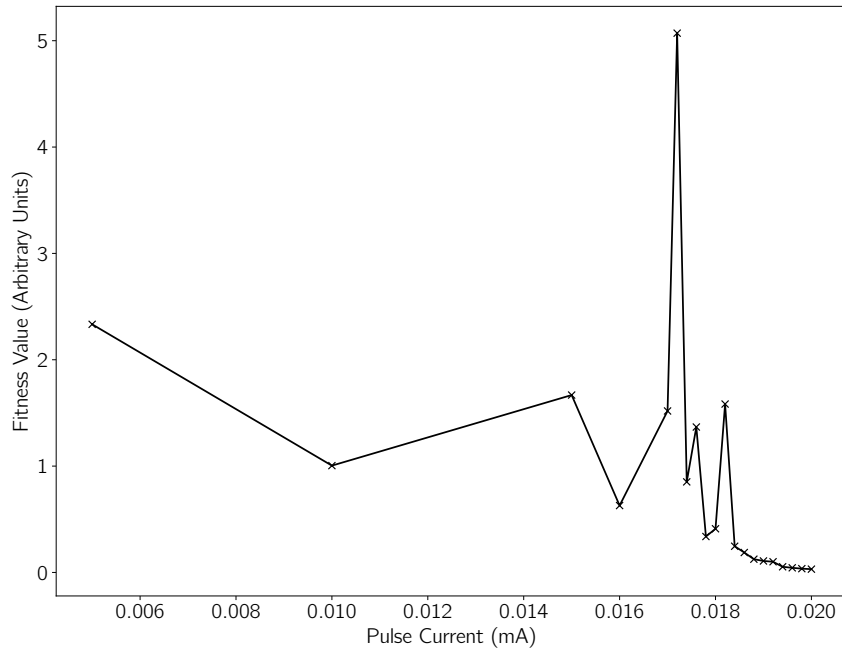
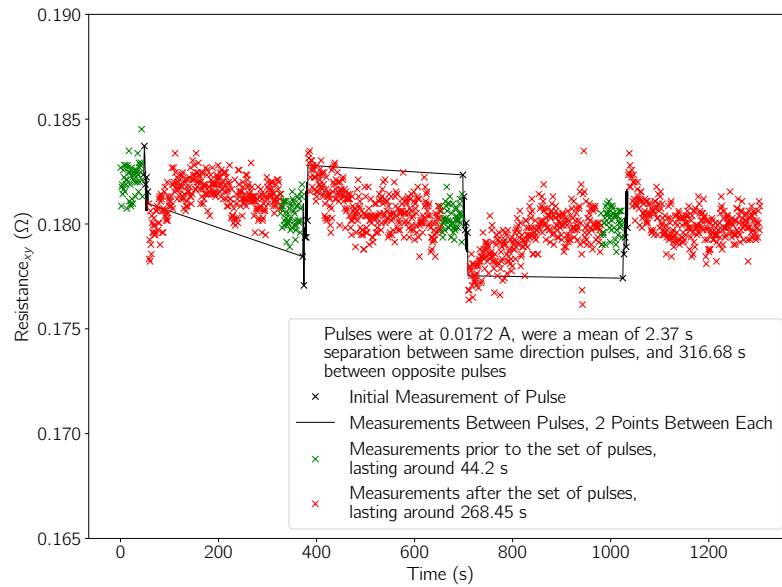


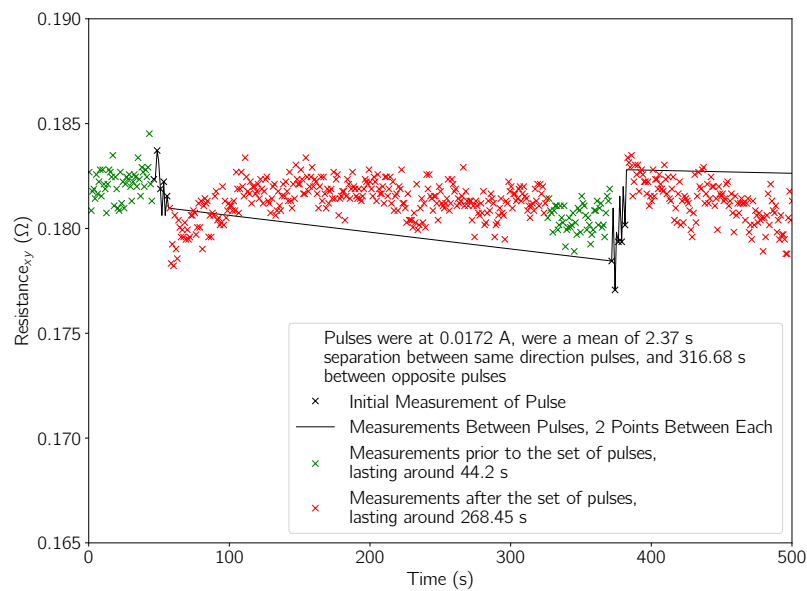
Figure 14: This shows fitness values plotted against the pulse current.

Figure 14 shows the result of this. The value drops and settles at around 18.4 mA, which is the point at which the switching behavior becomes more apparent. Hence the threshold current found during the investigation on the CuMnAs chip was 18.4 mA.

3.2.2 Multilevel Nature



(a) This shows the full range of data around the four switching sections.

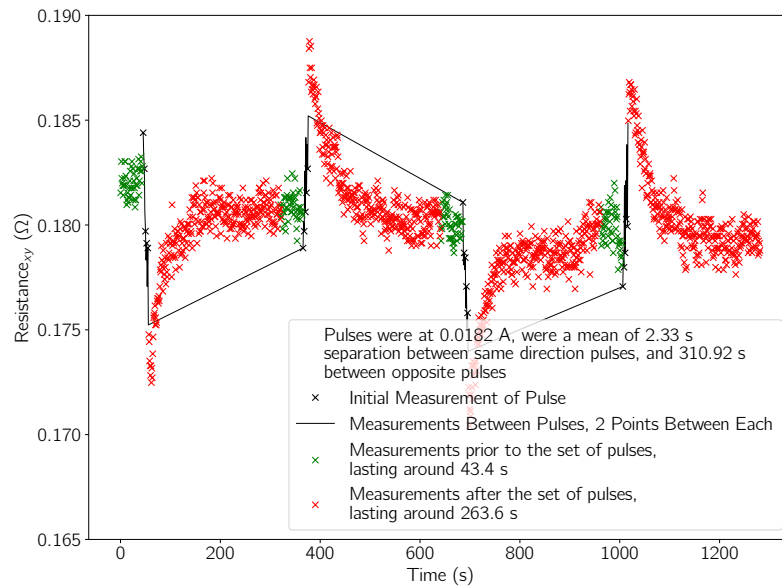


(b) This shows a specific range of data surrounding the first two switching sections.

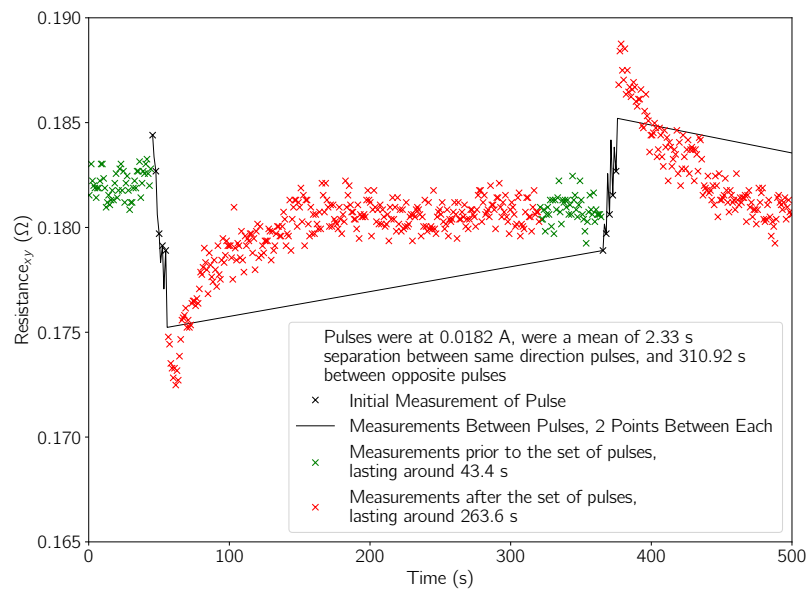
Figure 15: These graphs show the multilevel switching at 17.2 mA, there were two positive switching and two negative switching sections, each pulsed five times.



Figure 15 shows the multilevel switching that occurs at 17.2 mA. The separation and distinction between the levels is not clear.



(a) This shows the full range of data around the four switching sections.

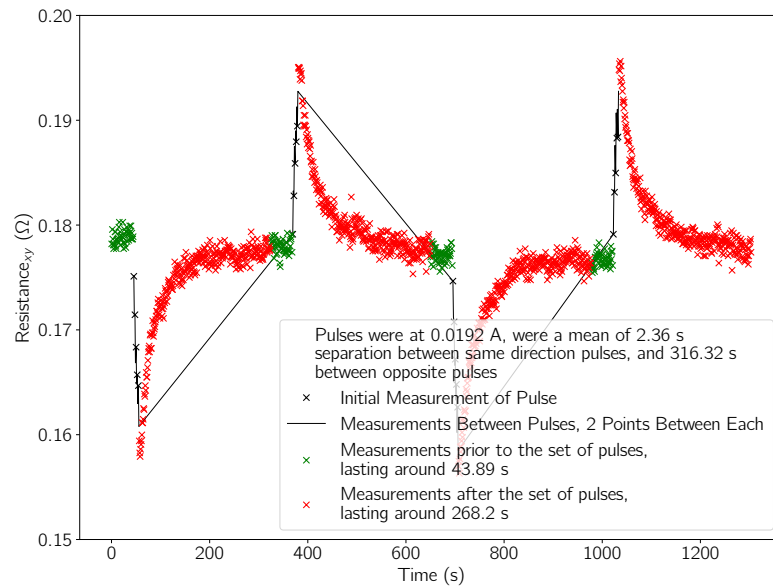


(b) This shows a specific range of data surrounding the first two switching sections.

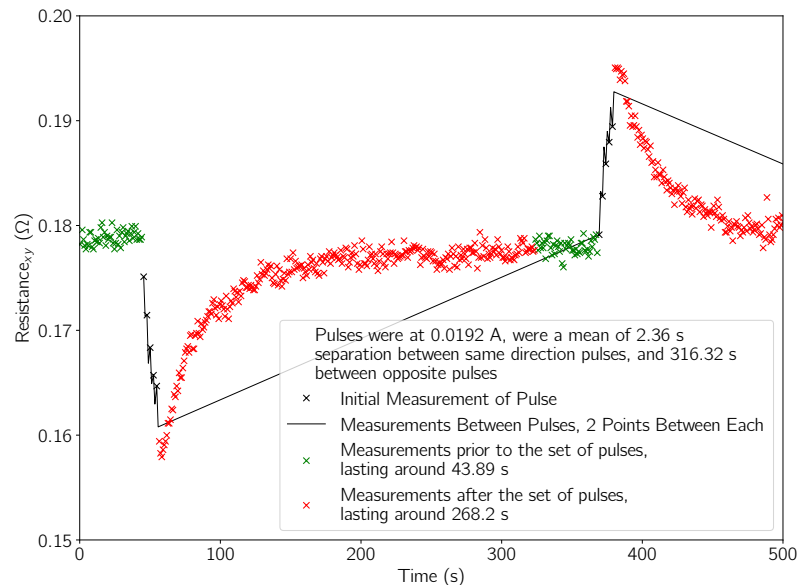
Figure 16: These graphs show the multilevel switching at 18.2 mA, there were two positive switching and two negative switching sections, each pulsed five times.



Figure 16 shows the multilevel switching that occurs at 18.2 mA. The levels are more clear and distinct than those at 17.2 mA, but the levels in the middle of the five pulses are less clearly separated.



(a) This shows the full range of data around the four switching sections.



(b) This shows a specific range of data surrounding the first two switching sections.

Figure 17: These graphs show the multilevel switching at 19.2 mA, there were two positive switching and two negative switching sections, each pulsed five times.



Figure 17 shows the multilevel switching that occurs at 19.2 mA. The levels are yet more clear and distinct than previously. This is expected with a higher pulse current.

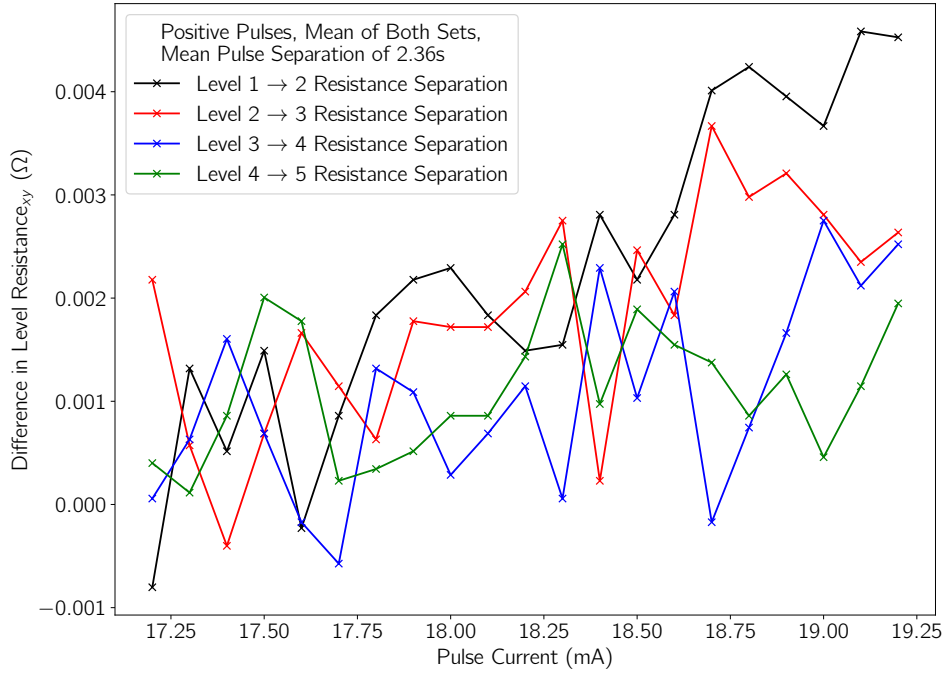


Figure 18: This graph shows the effects of the amplitude of the writing pulses in direction H $\rightarrow$ D on the resistance between levels. A positive trend in the difference in resistance can be seen. The separation time between pulses for all measurements had a mean time of 2.36 s.

In figure 18 we observe the positive correlation between the change in resistance levels as the amplitude of the writing pulses was increased from 17.3 mA to 19.1 mA along the input of H $\rightarrow$ D. In the beginning there was no ordering in the change of resistance after each pulse. Ordering began to take place after the threshold current, 18.45 mA, at this current amplitude switching was proven to take place. After this point the resistance jumps ordered into the expected format, as seen in the literature [29], the initial jump being the largest and the final jump being the smallest, for a discussion on the expected format view section 4.2.2.

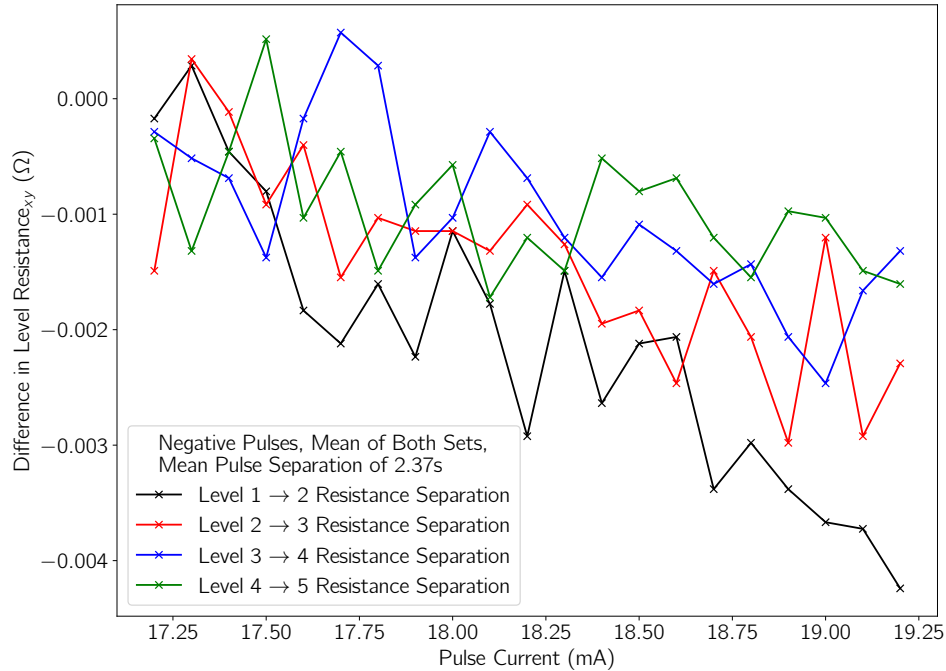
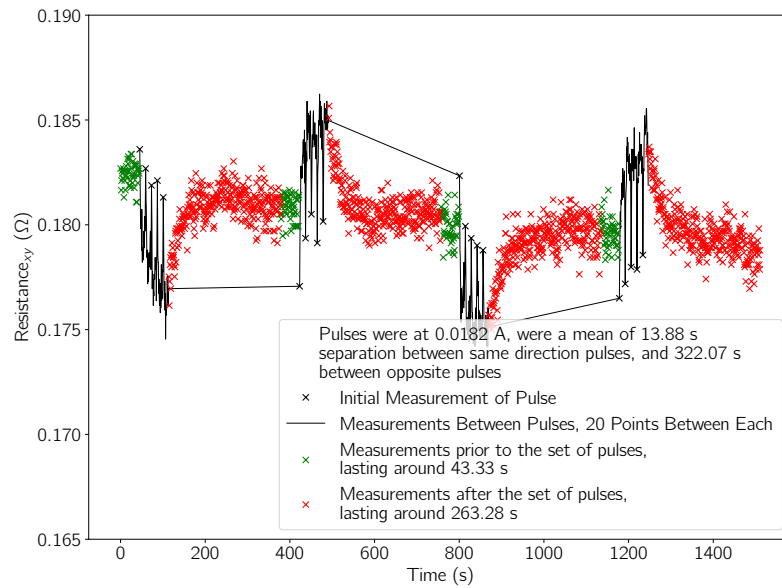


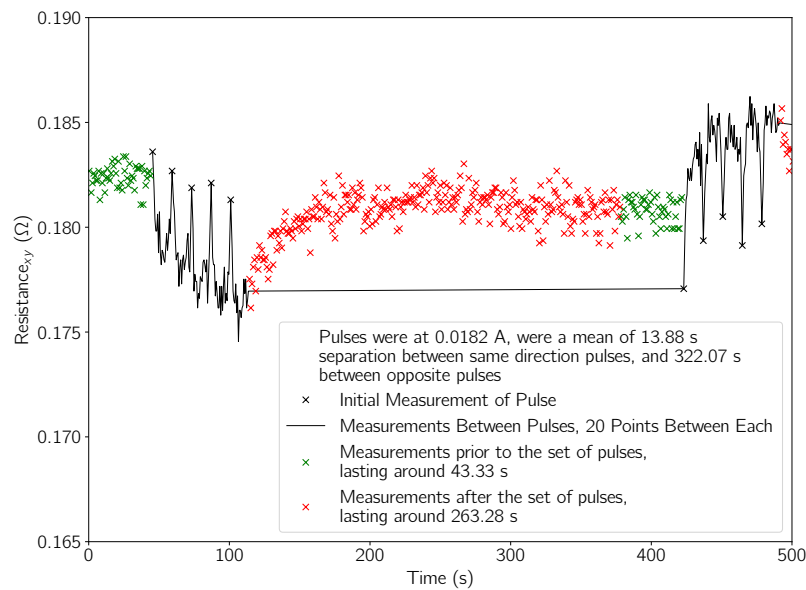
Figure 19: This graph shows the effects of the amplitude of the writing pulses in direction B $\rightarrow$ F on the resistance between levels. A negative trend in the difference in resistance can be seen. The separation time between pulses for all measurements had a mean time of 2.37 s.

In figure 19 we observe the negative correlation between the change in resistance levels as the amplitude of the writing pulses was increased from 17.2 mA to 19.2 mA along the input of B $\rightarrow$ F. In the beginning there was no ordering in the change of resistance after each pulse. Much like figure 18 the ordering began after the threshold current was reached. After this point the resistance jumps ordered into the expected format, however, there was an unexpected observable instability in the red and blue lines, as they cross frequently.

When investigating the effect of separation time between writing pulses and the change in resistance in the steps, a constant writing current was used. The current was kept at 18.2 mA, this protected the chip from decay as it was slightly before the true threshold but it was also close enough that successive pulses would result in switching.



(a) This shows the full range of data around the four switching sections.

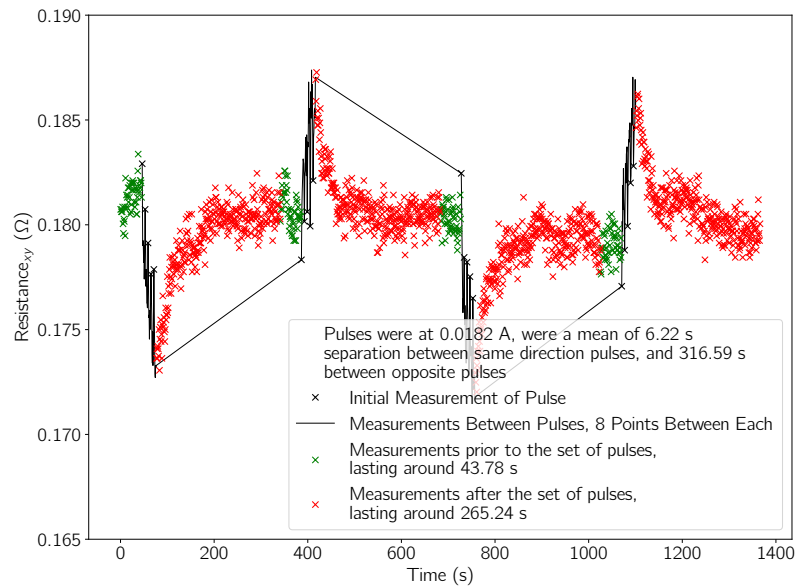


(b) This shows a specific range of data surrounding the first two switching sections.

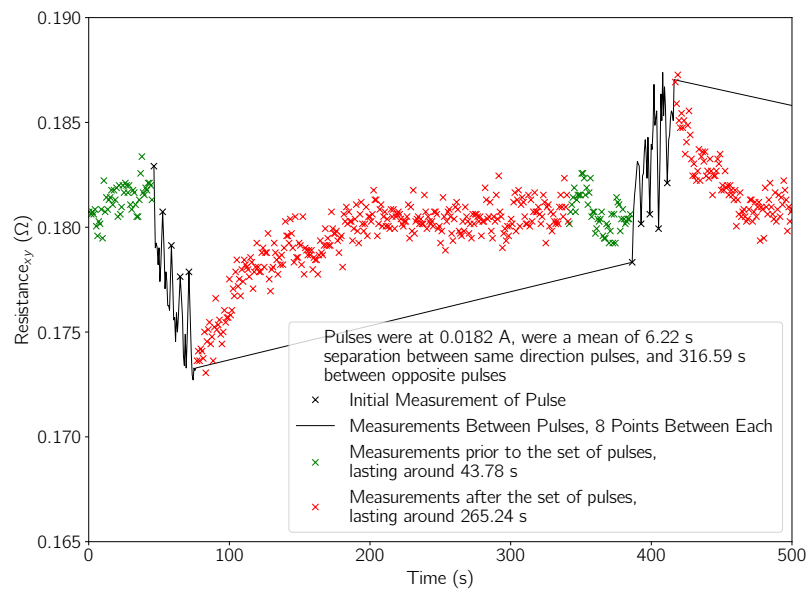
Figure 20: These graphs show the multilevel switching with a pulse separation of 13.88 s, there were two positive switching and two negative switching sections, each pulsed five times.

Figure 20 shows the multilevel switching that occurs with a pulse separation of 13.88 s. The levels are not clearly distinct from each other, which is expected with a longer time separation as it allows time for the chip to settle down.





(a) This shows the full range of data around the four switching sections.



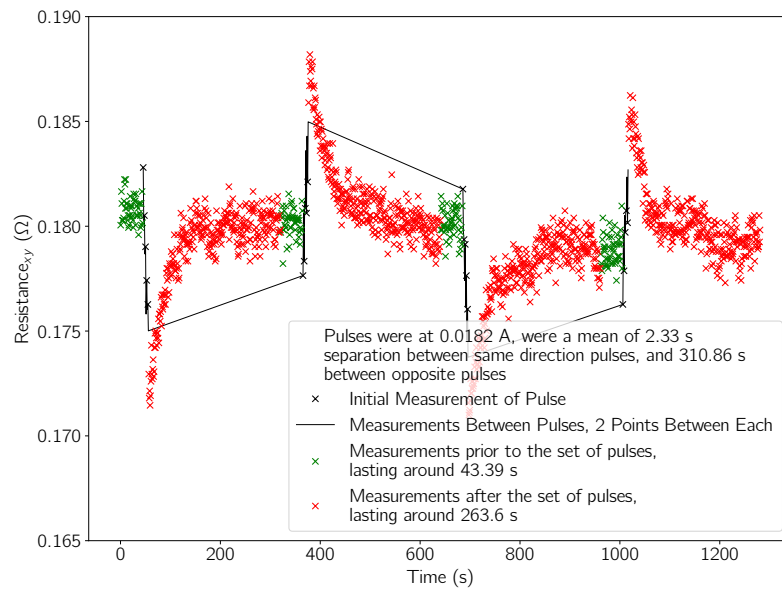
(b) This shows a specific range of data surrounding the first two switching sections.

Figure 21: These graphs show the multilevel switching with a pulse separation of 6.22 s, there were two positive switching and two negative switching sections, each pulsed five times.

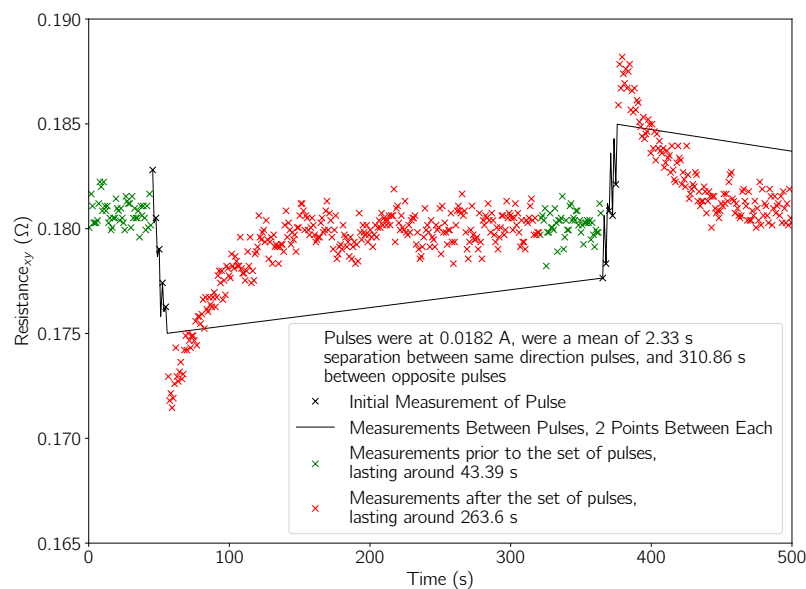
Figure 21 shows the multilevel switching that occurs with a pulse separation of 6.22 s. The levels are clearer than previous tests, but still not very clear as some have around the same values in the middle. This is expected, as while the separation is smaller, it is not small enough



to consistently push the resistance up past the previous level.



(a) This shows the full range of data around the four switching sections.



(b) This shows a specific range of data surrounding the first two switching sections.

Figure 22: These graphs show the multilevel switching with a pulse separation of 2.33 s, there were two positive switching and two negative switching sections, each pulsed five times.

In figure 22, the multilevel switching occurs with a separation of 2.33 s. This allows



successive pulses to push up the resistance past what it was previously. This means that clear and distinct levels can be observed.

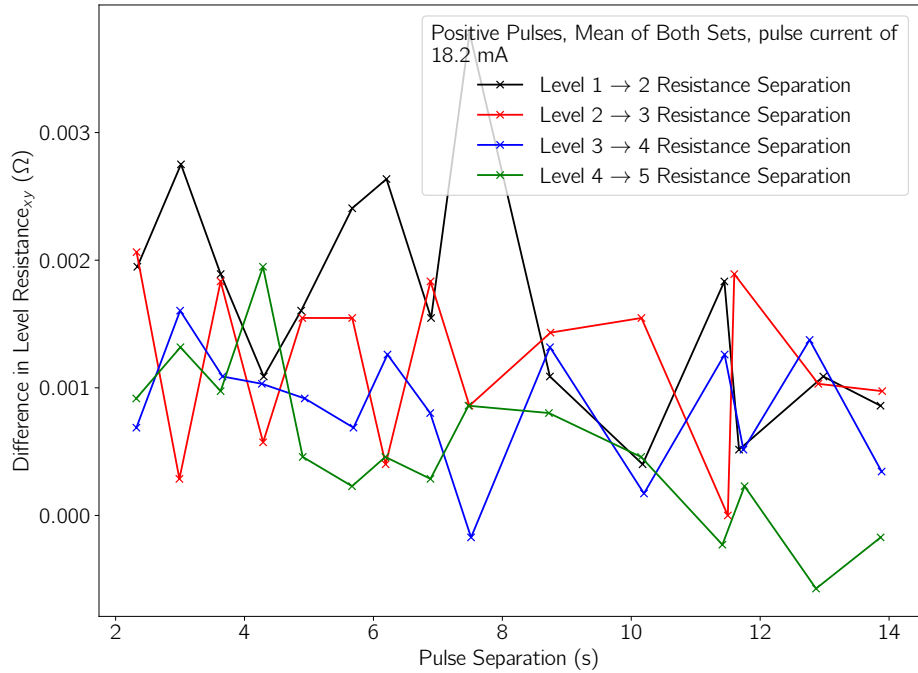


Figure 23: This graph shows the effects of the separation time of the writing pulses in direction $H \rightarrow D$ on the resistance between levels. In this image there is no clear trend. The amplitude of the pulses was fixed for all measurements at 18.2 mA.

In figure 23 we observe the correlation between the change in resistance levels as the separation time of the writing pulses was increased from 2 s to 14 s along the writing pulse channel of $H \rightarrow D$. The data fluctuates at an extremely low resistance and has a very weak trend from a positive resistance towards 0 Ω . The very low resistance and trend in comparison to the results shown in figures 18,19 and 24 leads to the conclusion that this graph provides no statistical evidence for a change in the resistance when the separation time of the writing pulse across $H \rightarrow D$ was varied. This result contrasts the results of the previous investigation; a comparison can be found in section 4.2.2.

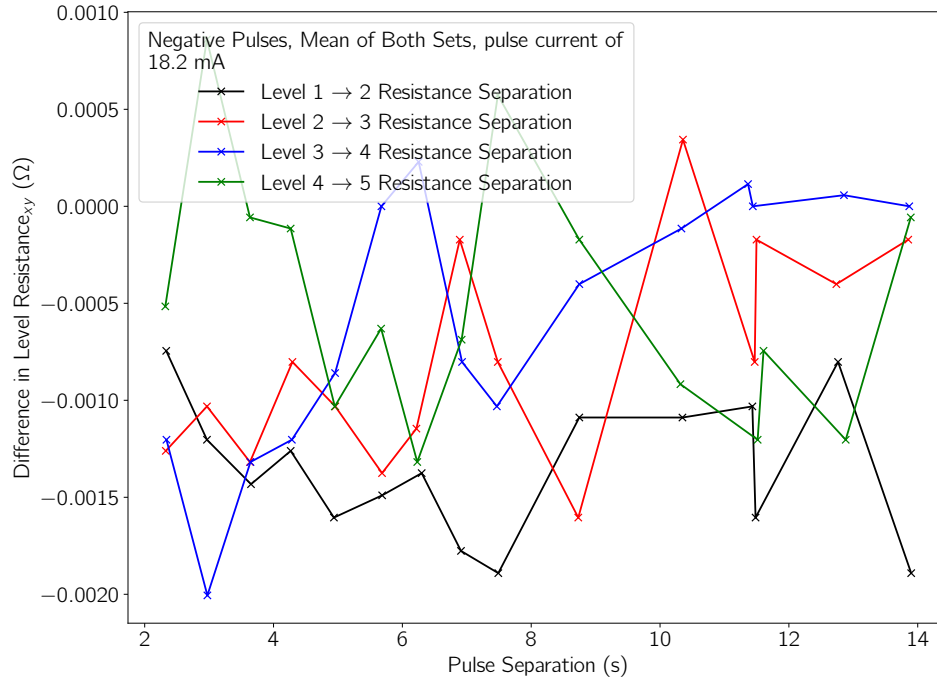


Figure 24: This graph shows the effects of the separation time of the writing pulses in direction $B \rightarrow F$ on the resistance between levels. In this image, there is a general trend towards no resistance change as the separation time increases. The amplitude of the pulses was fixed for all measurements at 18.2 mA.

In figure 24 we observe the weak positive correlation between the change in resistance levels as the separation time of the writing pulses was increased from 2 s to 14 s along the input of $B \rightarrow F$. Starting at a resistance averaging around $1.5\text{m}\Omega$ the plot gradually increases over the pulse separation time until it reaches 0Ω . This can be explained by the exponential decay of the resistance as seen in figure 10. The positive correlation matches the expected outcome seen in the literature [29].

4 Discussion of Results

4.1 Modelling

Literature has shown the Monte Carlo method to be an excellent avenue to approach the issue of creating a simulation of an antiferromagnet, this is further supported by the successful production of this investigation's model. Papers such as Máca *et al.* [30] have used this method to calculate the exchange energies present in CuMnAs in the past, in which they used the data obtained from the model to estimate the material's Néel temperature and transport properties via the TB-LMTO-CPA approach. Despite the success of the simulation in producing an animation of an infinitely looped sheet of antiferromagnetic material, there were many road-blocks present in the construction of said model.

The first limitation present within the model is the restriction of the antiferromagnet to a singular layer. This restricts the model of the moments to a two-dimensional plane. The overarching issue with the two-dimensional interpretation of the magnetic moment in the single layer is that no out-of-plane component is considered. While events like the Néel domain wall are capable of forming within this system; events like the Bloch domain wall are incapable of being modelled in this simulation. This also limits the advancement of the model; any anisotropic magnetoresistance analysis would be limited to the material's plane thus phenomena such as the anomalous Hall effect would not be modelled [23].

In addition to the previous issues, this error transitions into a distorting of the energy values present in the magnet. The introduced layers above and below would each provide a single nearest neighbour moment to every moment in the original layer. It would also increase the next nearest neighbour count by four. The increased accuracy would be an inviting prospect for the simulation as the next nearest neighbours have a notable effect on the moment energy [31]. However, the energy calculations are limited to only the nearest neighbours. This self-imposed limitation was introduced at the detriment to the simulation's accuracy so that the simulation would complete its calculations within a suitable time frame.

Improvements made in the process of creating the simulation, such as the addition of edge looping caused a marked change in the output animations. The fixed edges limited the disorder at the start and introduced a locking effect on the neighbouring moments which reduced the ability of the system to vary between separate runs of the simulation. Despite removing the issues of the fixed edging, the looping system was also fallible. In most of the simulation's runs a circular domain wall would form in the corners, as seen in figure 8, with each corner forming a quarter circle around it. A circular domain wall encompassing an area of magnetic order would be unlikely to form in a physical magnet because of the unfavorably high energy cost associated with this structure [32].

To advance the model, it would be advisable to switch from iteration-by-iteration model to a time-wise model. This is because a time-wise model is more easily comparable to real magnets than an iteration-by-iteration method. A time-wise model provides information on the speed of the changes calculated rather than the iteration-by-iteration model that is not framed in a way conducive to making comparisons in real life, not easily accounting for how fast this process is, thus acting as a worse representation of the timeline and real-world reactions.

Further computations could be added to the simulation, the purpose of this, to act as an analogue for the physical CuMnAs sample under electrical switching. The additional calculations required for this would first be the introduction of a writing process. A writing pulse could be equated to a magnetic field passing through the sample, acting as a simplified version



of the Néel spin-orbit torque described in section 1.2.2. The associated relaxation time of the moments would also be included, along with the switch to a second-by-second model, which would result in a suitable computational stand-in for switching in CuMnAs. This could be taken another step with the resistance tests used in the physical experimentation, a set of calculations for the anisotropic magnetoresistance could be introduced into the simulation. However, all these improvements require an accurate account of the parameters needed, for example, the non-crystalline coefficient (C_{NC}) would be needed to calculate the anisotropic magnetoresistance.

4.2 Experimental Discussion

As both the threshold current investigation and multilevel investigation share the same equipment, they face many of the same issues. An ever-present issue within the experimental readings was noise. Shot noise was present within the readings as it is the random statistical fluctuation of the current and is associated with all electrical devices. Another issue faced by the investigation was the vibrational noise in the readings. This was reduced as much as possible; an example of vibrational noise prevention is the removal of human contact with any equipment, other than the computer controls. However, the vibrations from the floor still affect the table and hence the housing for the chip. Fortunately, the vibrational noise that the housing experienced was quite small.

Johnson–Nyquist noise was likely a contributing factor due to the variation of temperature in the laboratory. The air temperature around the chip was not controlled so can vary by noticeable amounts within working hours in the laboratory. This error is why research completed by Wadley *et al.* [11] at the University of Nottingham was completed in a temperature controlled laboratory.

Finally, there will be a shadowing effect that increased the noise. This is due to small offsets in the input and output channels as well as the spread of current to surrounding CuMnAs material. This spreading and change in current density can be observed in figure 27. At the entrances of the unused conduction channels, a current density 6 - 12 % the size of the injected current was experienced [15]. The figure illustrates the increased resistance of the chip as areas experience the Néel spin-orbit torque reorienting moments into higher resistance states for reading.

A so far unresolved error faced across both experiments was the initial starting state of the CuMnAs chip. The chip would always start with magnetic moments in inconsistent orientations. This affected the decay curves produced in section 3.2.1 which had varying starting positions (the magnetic switching would vary in the number of reorientated moments). As well as being the main cause of the unexpected fluctuation of resistance in section 3.2.2, it would be expected that tests at a set current would produce repeatable outputs with an identical chip. A solution to this issue would be to introduce a resetting cycle at the beginning and end of each test. This would result in a semi-uniform starting position for every test hence limiting the variation between tests.

While these factors are important to consider when talking about variations in individual readings, the resistance is shown to be consistent across the experimentation on the chip. A point demonstrated by figure 26. This has the additional purpose of explicitly illustrating that no permanent decay hindered the readings from the chip.



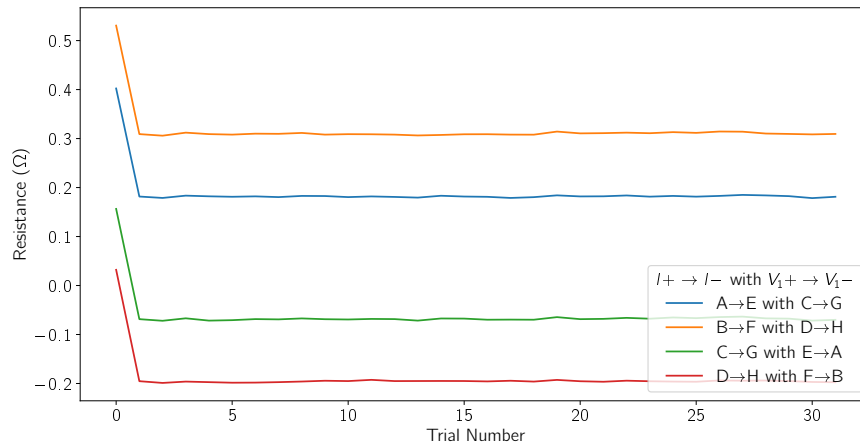


Figure 25: Resistance measurements taken before our trials doing single level switching, plotted against trial number.

Figure 25 shows consistent resistance levels across most trials, and more importantly, across the trial range we used for our data, trial numbers 11 to 31².

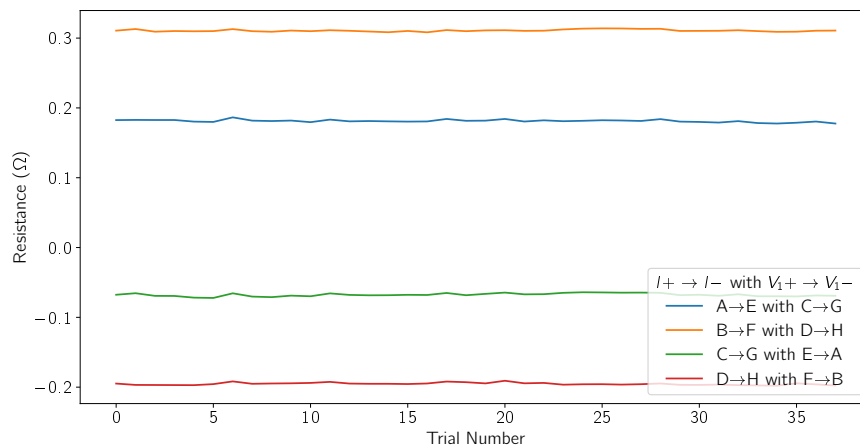


Figure 26: Resistance measurements were taken before our trials doing multilevel switching, plotted against the trial number.

Figure 26 shows consistent resistance levels across all trials, and more importantly across the ranges we used, trial numbers 2-16 for our pulse separation variation data, and trial numbers 18-38 for our pulse current variation data.

²Excluding 14 as this is the first time we ran 20 mA and found it switched, and so it would be a duplicate result of trial 31, which was also run at 20 mA.



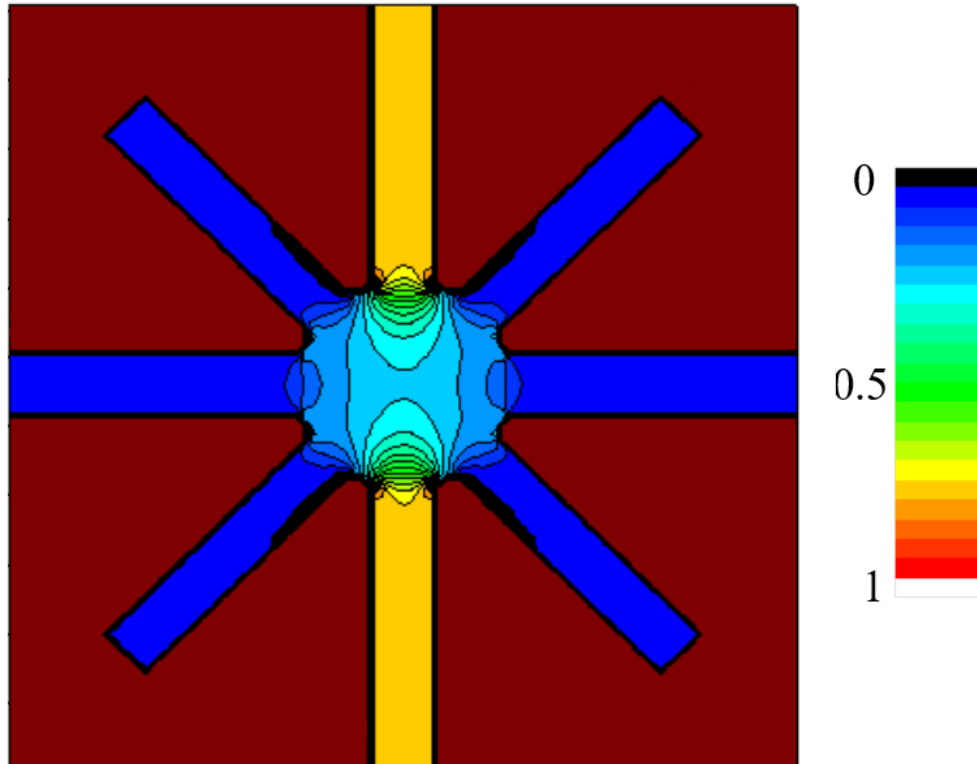


Figure 27: This image illustrates the spread of current density along the y-axis when a writing pulse passes from the top conductive channel, into the CuMnAs centre, and then out through the bottom conductive channel. The figure was produced by Howells [15].

As a writing pulse is sent through the chip the time taken for moments within the CuMnAs to reorient changes, as it depends on the magnitude of the field-like torque that each moment experiences. The issue arises when a reading attempt is initiated. The time difference between the first and last moments reorienting is unknown. If a reading pulse is sent too early, it would miss the resistance generated by the final moment. Counter to this, if the reading pulse is sent too late, the first moments to reorient start to decay. The ideal situation was to measure the resistance at its peak, where the balance of missing moments and limiting decay met. This is a difficult aim to achieve. The investigation instead used a fixed time at which the peak was estimated to occur. However, as the position of the peak could fluctuate the estimation was only broadly accurate, so the sampling of the resistance is inconsistent when looking for the peak resistance generated by the writing pulses. This is another issue faced in determining the resistance difference between levels in the multilevel investigation. There is also an important consideration with regards to small temperature changes, because of equation 9. If there is a decrease in temperature, the decay time increases for the moments affected by the writing pulse. In addition when there is an increase in temperature, the decay time decrease for the moments. This could have an effect on the tail/lead time that we see in figure 11.

Another issue that spans the experimental portion of this investigation is the lack of repeat measurements. The main cause of this issue was the need to replace chips. Power surges through coding issues connected with the switchbox as well as large current accidentally being passed through due to user error resulted in chips breaking. At final count, the chip was

replaced two times, and pins needed reconnecting on another two occasions. These delays prevented further advancement of the writing/reading code and limited the number of repeats completed for each aim, resulting in less accurate data.

4.2.1 Threshold Current

The threshold current was initially determined by visually inspecting the results and determining when the chip began to switch by recognising the characteristic decay curve. This method allowed for quick recognition of the threshold current but was inherently inconsistent. For this chip the threshold was found to be around 18-18.2 mA. With this chip, the data collected was consistent with what was expected, especially regarding the decay curves (seen in red in the graphs). Figure 10 shows the graph obtained when using an insufficient pulse current, and looks as expected. In figure 11, there was something odd that occurred however with the initial measurements (seen in black). They were on the opposite side to what was expected. This was investigated and found to be due to a lead time on the resistance reacting to the pulse current. This was suggested to be the result of the measurement to be a sort of ringing of the prior probe current, or something else.

In figure 12 we can see that the pulse current has increased far enough to get a consistent decay curve. In the first investigation of this particular chip, the pulse current was increased from 5 mA in 5 mA steps until hitting 20^3 , as the clear decay curve was observed. Once this was observed, a reading was taken at 16 mA, 17 mA, and then from there to 20 mA in 0.2 mA steps. This allowed the visual inspection fine-tuning to find the decay curve.

When looking through the various curves observed, it was realised that it would be possible to fit the data from the individual decay curves with the Kohlrausch stretched exponential function found in the paper by Kašpar *et al.* [26]. This would allow for accurate determination of the threshold current, as when the decay curves were properly forming, the covariances of the fitting parameters would be lower than when the data was more spread out, and hence not forming. The covariances were combined into one value, which could be called “fitness value”⁴. The method used was not specific but was fair. First, the covariances of the parameters were square rooted to get the 1 standard deviation error on the parameters (this step is not redundant as they are needed when stating the parameter result). These errors were divided by their respective parameter values to get a relative error. These relative errors were squared, then they were averaged by way of what pulse current they referred to, and finally square-rooted. This “fitness value” would decrease as the fit got better, but more importantly, it would experience a drop and steadying out immediately when the threshold current was hit. This method is advantageous over simply looking as it is repeatable and theoretically the point at which the data dips could be detected via computer⁵. This means that this technique could be used to speed up the testing of these devices or theoretically, with significant development, could be used in a factory to assess antiferromagnetic chips on an industrial scale. Unfortunately the curve fitting used⁶ returned a lot of unable to fit errors, or large unrealistic covariances. To counter this in the code, these larger numbers were set to NaN, so their value could be filtered out when collating the covariances into the “fitness value”. Using this method, we

³It was done this quickly in order to speed up the finding of the threshold current as the previous chip was broken with limited time left. The previous chip was taken from 0 mA to 40 mA in 0.5 mA steps, and then further from that in even smaller steps.

⁴Though should not be compared to other literature “fitness values”

⁵Time constraints meant this could not be investigated, but this should be relatively easy to accomplish.

⁶`scipy.optimize.curve_fit`, with adapted starting parameters.



found the threshold current to be around 18.4 mA. The key element of this method is that theoretically, it will always pick the same shape of curve and fitness, so will be consistent in its selection between different chips. If time constraints allowed, an error would also be able to be estimated on the 18.4 mA, but it is almost certainly governed by the resolution of steps made in the range of the pulse current.

A benefit of using this method is that it also finds the parameters of the decay curve. Two of those parameters found, a , and c , as seen in equation 10, can't be compared to literature values very easily. The a parameter scales with the height of the curve, and so scales with the size of the pulse current mainly. The c parameter is the offset and is essentially linked to where the resistance decays to, which is essentially the base resistance. These two parameters have too much variability between different parameters. The b parameter is the time decay, τ_1 , found in equation 8 from the paper by Kašpar *et al.* [26]. The value found for τ_1 can be compared to the approximate values theorised in the paper by Olejník *et al.* [29] that it would be on the order of seconds. The value found, 22.5 ± 0.3 s, is consistent with this prediction.

4.2.2 Multilevel

The first portion of the experimentation into the multilevel nature of CuMnAs incrementally increased the magnitude of the writing current, the experiment has yielded data that corroborates the expected results seen in literature: Olejník *et al.* [29]. Figure 18 and figure 19 both demonstrate that once the threshold current is reached, a series of writing pulses when passed through the sample will cause a change in resistance after each pulse, demonstrating multiple levels of resistance. The ordering of the resistance jumps between pulses also matches the expected order seen in the literature. The ordering of the resistance jumps between pulses also matches the expected order seen in the literature. The multilevel readout from Olejník *et al.* [29] is shown in figure 28, in which the difference between levels was similar to those found in this investigation. It was expected that the first jump would be the largest and the last jump would be the smallest. Despite the fluctuations between the red and blue lines in both graphs, which are noise related, the graphs demonstrate clear ordering within the system. If more repeats were completed these fluctuation would be averaged out, the addition of error bars would clarify to the variation. However, with only one repeat measurement being taken for both figure 18 and figure 19 the error bars would not accurately provide an error range.



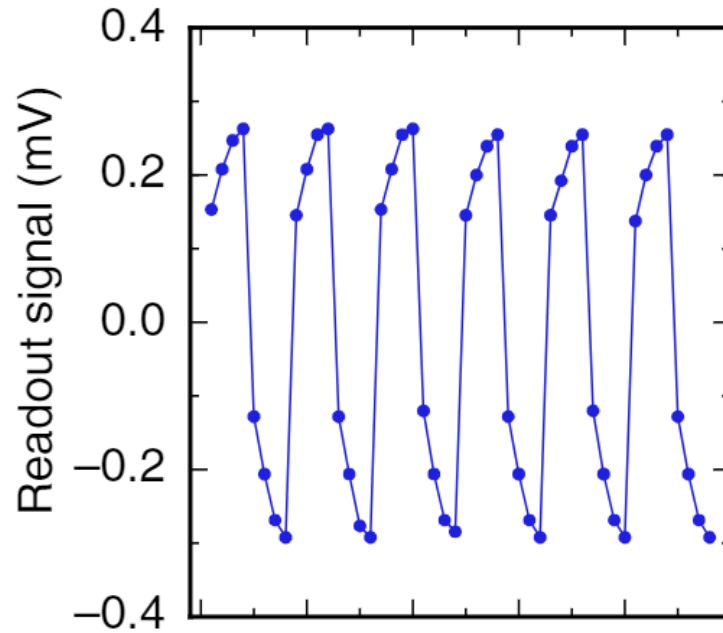


Figure 28: This image illustrates the multiple resistance levels achieved through experimentation on a CuMnAs chip. The difference between levels was comparable to those found in our investigation. The figure was produced by Olejník *et al.* [29] and each blue dot represents a reading after on writing pulse.

In addition to confirming the multilevel nature once the current is raised past the threshold and increased to higher values the correlation between the increased current and increased resistance change is evident. These results match the expected data trends. Literature has demonstrated that fixing the current density delivered and varying the time results in the same outcome Olejník *et al.* [29]. In that investigation, the longer time with fixed current density produced a larger resistance readout, the equivalent of a larger current over a fixed time in our experiment, as more electrons are passed through the sample. The result of the varied pulse duration with a fixed current density can be seen in figure 29.

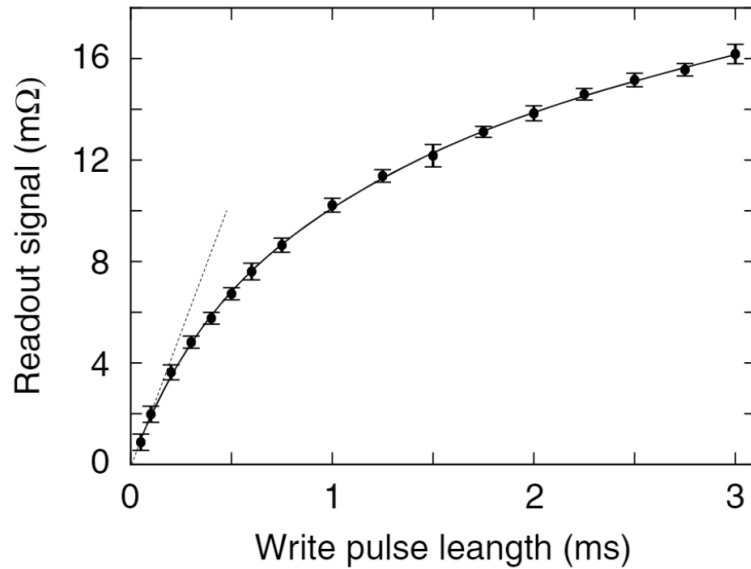


Figure 29: This graph illustrates the relationship when fixing the current density and varying the time of a pulse through experimentation on a CuMnAs chip. The figure was produced by Olejník *et al.* [29].

The second portion of the investigation into the multilevel nature of CuMnAs tested the effects of increasing the separation time on the difference in resistance levels of the chip. This investigation yielded very little result, with figure 23 showing no correlation. However, the data shown in figure 23 demonstrated a weak correlation. The trend found within this figure matched the expected result found in the literature [29]. The literature describes that an increased separation time between pulses will see a lower separation resistance between levels. This is due to the decay experienced by the resistance and described by section 3.2.1. Overall, that data from the experimentation on the separation time yielded unsatisfactory results. No conclusion can be drawn from them.

5 Conclusions

In summary, the investigation achieved many of its intended criteria. First, the simulation of an antiferromagnet was successful, possessing the ability to order from and disordered state. Despite the simplicity of the model, it presents itself as a reliable stepping stone for more complex applications to be adjoined. On the experimental side of the investigation, a melange of results was achieved. The threshold current sample was successfully found to be 18.4 mA, with a high confidence fit. In addition, the relaxation time of the sample was found at 22.5 ± 0.3 s, this supported the theory by Olejník *et al.* [29] that it would be in the order of seconds. The final prosperous task was the experimentation on the effect of an increased current on the CuMnAs' multilevel nature. The results of these tests demonstrated the expected multilevel spread of resistance, as well as an increased current led to an increased jump in resistance steps in the sample, a result supported by surrounding research. The final test was the experimentation of increased separation time between writing pulses on the multilevel nature of CuMnAs. This yielded unsatisfactory results, despite seeing a slight trend in favour of decreased separation time for increased resistance jumps, no well-founded conclusion can be drawn from these results.



References

1. Fiebig, M. *et al.* Ultrafast magnetization dynamics of antiferromagnetic compounds. *Journal of Physics D: Applied Physics* **41**, 164005. <https://doi.org/10.1088/0022-3727/41/16/164005> (Aug. 2008).
2. Kimel, A. V., Kirilyuk, A., Tsvetkov, A., Pisarev, R. V. & Rasing, T. Laser-induced ultrafast spin reorientation in the antiferromagnet TmFeO₃. *Nature* **429**, 850–853. ISSN: 0028-0836 (2004).
3. Marti, X. *et al.* Room-temperature antiferromagnetic memory resistor. *Nature Materials* **13**, 367–374. ISSN: 1476-1122. <https://dx.doi.org/10.1038/nmat3861> (2014).
4. Němec, P., Fiebig, M., Kampfrath, T. & Kimel, A. V. Antiferromagnetic opto-spintronics. *Nature Physics* **14**, 229–241. ISSN: 1745-2473 (2018).
5. Nordrum, A. *Are Antiferromagnets the Next Step for MRAM?* Available at: <https://spectrum.ieee.org/nanoclast/semiconductors/memory/antiferromagnets-next-step-mram>. May 2020.
6. Marrows, C. Addressing an antiferromagnetic memory. *Science* **351**, 558–559. ISSN: 0036-8075 (2016).
7. Gomonay, O., Jungwirth, T. & Sinova, J. Concepts of antiferromagnetic spintronics. *physica status solidi (RRL) – Rapid Research Letters* **11**, 1700022. ISSN: 1862-6254. <https://dx.doi.org/10.1002/pssr.201700022> (2017).
8. Jungwirth, T., Marti, X., Wadley, P. & Wunderlich, J. Antiferromagnetic spintronics. *Nature Nanotechnology* **11**, 231–241. ISSN: 1748-3387. <https://dx.doi.org/10.1038/nnano.2016.18> (2016).
9. Olejník, K. *et al.* Terahertz electrical writing speed in an antiferromagnetic memory. *Science Advances* **4**, eaar3566. ISSN: 2375-2548. <https://dx.doi.org/10.1126/sciadv.aar3566> (2018).
10. Liu, L., Lee, O. J., Gudmundsen, T. J., Ralph, D. C. & Buhrman, R. A. Current-Induced Switching of Perpendicularly Magnetized Magnetic Layers Using Spin Torque from the Spin Hall Effect. *Physical Review Letters* **109**. ISSN: 0031-9007 (2012).
11. Wadley, P. *et al.* Tetragonal phase of epitaxial room-temperature antiferromagnet CuMnAs. *Nature Communications* **4**. ISSN: 2041-1723. <https://dx.doi.org/10.1038/ncomms3322> (2013).
12. Železný, J. *et al.* Relativistic Néel-Order Fields Induced by Electrical Current in Antiferromagnets. *Phys. Rev. Lett.* **113**, 157201. <https://link.aps.org/doi/10.1103/PhysRevLett.113.157201> (15 Oct. 2014).
13. Wadley, P. *et al.* Electrical switching of an antiferromagnet. *Science* **351**, 587–590. ISSN: 0036-8075. <https://dx.doi.org/10.1126/science.aab1031> (2016).
14. Bodnar, S. Y. *et al.* Writing and reading antiferromagnetic Mn₂Au by Néel spin-orbit torques and large anisotropic magnetoresistance. *Nature Communications* **9**. ISSN: 2041-1723. <https://dx.doi.org/10.1038/s41467-017-02780-x> (2018).
15. Howells, B. *Studies of spin-orbit coupling phenomena in magnetic semiconductors* July 2015. <http://eprints.nottingham.ac.uk/29005/>.



16. Skinner, T. D. *et al.* Complementary spin-Hall and inverse spin-galvanic effect torques in a ferromagnet/semiconductor bilayer. *Nature Communications* **6**, 6730. ISSN: 2041-1723. <https://dx.doi.org/10.1038/ncomms7730> (2015).
17. Yuan, L. *et al.* Uncovering and tailoring hidden Rashba spin-orbit splitting in centrosymmetric crystals. *Nature Communications* **10**. ISSN: 2041-1723. <https://dx.doi.org/10.1038/s41467-019-08836-4> (2019).
18. Zhang, X., Liu, Q., Luo, J.-W., Freeman, A. J. & Zunger, A. Hidden spin polarization in inversion-symmetric bulk crystals. *Nature Physics* **10**, 387–393. ISSN: 1745-2473. <https://dx.doi.org/10.1038/nphys2933> (2014).
19. Xie, Z. *et al.* Orbital-selective spin texture and its manipulation in a topological insulator. *Nature Communications* **5**. ISSN: 2041-1723. <https://dx.doi.org/10.1038/ncomms4382> (2014).
20. Železný, J., Wadley, P., Olejník, K., Hoffmann, A. & Ohno, H. Spin transport and spin torque in antiferromagnetic devices. *Nature Physics* **14**, 220–228. ISSN: 1745-2473. <https://dx.doi.org/10.1038/s41567-018-0062-7> (2018).
21. Wang, C. *et al.* Anisotropic Magnetoresistance in Antiferromagnetic Sr2IrO4. *Physical Review X* **4**. ISSN: 2160-3308 (2014).
22. Zhao, C.-J., Ding, L., Huangfu, J.-S., Zhang, J.-Y. & Yu, G.-H. Research progress in anisotropic magnetoresistance. *Rare Metals* **32**, 213–224. ISSN: 1001-0521 (2013).
23. Shao, D.-F., Zhang, S.-H., Gurung, G., Yang, W. & Tsymbal, E. Y. Nonlinear Anomalous Hall Effect for Néel Vector Detection. *Physical Review Letters* **124**. ISSN: 0031-9007 (2020).
24. Rushforth, A. W. *et al.* Anisotropic Magnetoresistance Components in (Ga,Mn)As. *Physical Review Letters* **99**. ISSN: 0031-9007 (2007).
25. Rushforth, A. *et al.* The origin and control of the sources of AMR in (Ga,Mn)As devices. *Journal of Magnetism and Magnetic Materials* **321**, 1001–1008. ISSN: 0304-8853. <https://dx.doi.org/10.1016/j.jmmm.2008.04.070> (2009).
26. Kašpar, Z. *et al.* Quenching of an antiferromagnet into high resistivity states using electrical or ultrashort optical pulses. *Nature Electronics* **4**, 30–37. ISSN: 2520-1131 (2021).
27. Phillips, J. Axiomatic theories of ideal stretched exponential relaxation (SER). *Journal of Non-Crystalline Solids* **352**, 4490–4494. ISSN: 0022-3093. <https://dx.doi.org/10.1016/j.jnoncrysol.2006.01.162> (2006).
28. Wadley, P. *et al.* Current-polarity dependent manipulation of antiferromagnetic domains. *Nature Nanotechnology* **13** (May 2018).
29. Olejník, K. *et al.* Antiferromagnetic CuMnAs multi-level memory cell with microelectronic compatibility. *Nature Communications* **8**, 15434. ISSN: 2041-1723. <https://dx.doi.org/10.1038/ncomms15434> (2017).
30. Máca, F. *et al.* Physical properties of the tetragonal CuMnAs: A first-principles study. *Physical Review B* **96**. ISSN: 2469-9950 (2017).
31. Petrenko, O. A. & Mck. Paul, D. Classical Heisenberg antiferromagnet on a garnet lattice: A Monte Carlo simulation. *Physical Review B* **63**. ISSN: 0163-1829 (2000).



32. Mouritsen, O. G. & Praestgaard, E. Dynamical scaling, domain-growth kinetics, and domain-wall shapes of quenched two-dimensional anisotropicXYmodels. *Physical Review B* **38**, 2703–2714. ISSN: 0163-1829 (1988).



6 Appendix

Appendix A Experimental Code

A.1 Resistance Measurement and Graphing

A.1.1 Resistance Measure - resistMeasure.py

```
import time
import numpy as np
import matplotlib
import winsound
import instruments
import matplotlib.pyplot as plt
import sys
import serial
import pyvisa as visa
from tkinter import filedialog as dialog

two_wire_assignments = ({ "I+": "A", "I-": "E"}, # sets up
    the 2 wire assignments
    { "I+": "B", "I-": "F"},
    { "I+": "C", "I-": "G"},
    { "I+": "D", "I-": "H"}
)

four_wire_assignments = ({ "I+": "A", "I-": "E", "V1+": "B", "
    V1-": "D"}, # sets up the 4 wire assignments
    { "I+": "B", "I-": "F", "V1+": "C", "
        V1-": "E"},
    { "I+": "C", "I-": "G", "V1+": "D", "
        V1-": "F"},
    { "I+": "D", "I-": "H", "V1+": "E", "
        V1-": "G"},
    { "I+": "E", "I-": "A", "V1+": "F", "
        V1-": "H"},
    { "I+": "F", "I-": "B", "V1+": "G", "
        V1-": "A"},
    { "I+": "G", "I-": "C", "V1+": "H", "
        V1-": "B"},
    { "I+": "H", "I-": "D", "V1+": "A", "
        V1-": "C"},
    { "I+": "A", "I-": "E", "V1+": "C", "
        V1-": "G"},
    { "I+": "B", "I-": "F", "V1+": "D", "
        V1-": "H"},
```



```
        {"I+": "C", "I-": "G", "V1+": "E", "
          V1-": "A"},
        {"I+": "D", "I-": "H", "V1+": "F", "
          V1-": "B"}
    )

two_wires = np.zeros(len(two_wire_assignments)) # sets up
    the number of two wire measurements to assign to
four_wires = np.zeros(len(four_wire_assignments)) # sets up
    the number of four wire measurements to assign to
probe_current = 500e-6

sb = instruments.SwitchBox() # sets up the switch box object
pg = instruments.K2461() # sets up the pulse generator
    object

sb.connect(6) # connects to both of the devices
pg.connect()

for i in range(len(two_wire_assignments)): # loops through
    all of the two wire assignments
    sb.switch(two_wire_assignments[i]) # switches to the
        correct assignment
    time.sleep(0.1) # sleeps for 100ms to give time to the
        switchbox to switch
    pg.enable_2_wire_probe(probe_current, 2) # enables the 2
        wire probe
    time.sleep(0.5) # gives the previous line time to work
    c, v = pg.read_one() # measures the current and voltage
        across the device
    two_wires[i] = (v / c) # assigns the result to it's spot
    pg.disable_probe_current() # turns off the probe current
        before switching. not doing this broke our first
        device

for i in range(len(four_wire_assignments)): # basically does
    the same as above, only with the four wire assignments
    sb.switch(four_wire_assignments[i])
    time.sleep(0.5)
    pg.enable_4_wire_probe(probe_current)
    time.sleep(0.1)
    c, v = pg.read_one()
    four_wires[i] = v / c
    pg.disable_probe_current()

pg.close() # closes the connections to the devices
sb.close()
```



```

print('Two_Wire_Resistances:', two_wires)  # prints the
    results
print('Four_Wire_Resistances:', four_wires)

name = dialog.asksaveasfilename(title='Save')  # asks where
    to save the results

if name:  # this whole section deals with auto naming the
    files so it's as pain free as possible
    name = name.replace('_2wires', '')
    name = name.replace('_4wires', '')
    numberstr = name[-14:-11]
    number = int(numberstr)
    nextnumber = number + 1
    nexnumstr = f'{nextnumber:03d}'
    name = name.replace(numberstr, nexnumstr)
    name_two_wires = name.replace('.txt', '_2wires.txt')
    np.savetxt(name_two_wires, two_wires, newline='\n',
        delimiter='\t')  # save
    print(f'Two_Wires_data_saved_as_{name_two_wires}')

    name_four_wires = name.replace('.txt', '_4wires.txt')
    np.savetxt(name_four_wires, four_wires, newline='\n',
        delimiter='\t')  # save
    print(f'Four_Wires_data_saved_as_{name_four_wires}')
else:
    print('No_Filename:_Data_not_saved')

```

A.1.2 Resistance Graphing - resistGraph.py

```

import time
import numpy as np
import matplotlib as mpl
import winsound
import matplotlib.pyplot as plt
import sys
from tkinter import filedialog as dialog
import os

mpl.use("pgf")  # sets up mpl to use latex
mpl.rcParams['font.size'] = 18
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "font.family": "sans-serif",

```



```

        "pgf.preamble": [
            r"\usepackage[utf8x]{inputenc}",
            r"\usepackage[T1]{fontenc}",
            r"\usepackage{cmbright}",
        ]
    }

mpl.rcParams.update(style)

def stringsel(strlist, strch): # defines a function to grab
    strings containing another string out of a list
    res_strlist = []
    for i in range(len(strlist)):
        if strch in strlist[i]:
            res_strlist += [strlist[i]]
        else:
            pass
    return res_strlist

resist_dir = dialog.askdirectory() # asks for the directory
    of the place you put your resistance files from
resistMeasure.py
dir_list = os.listdir(resist_dir) # gets a list of all files
    in that list
wire2list = stringsel(dir_list, '2wires.txt') # grabs the 2
    wire resistance files
wire4list = stringsel(dir_list, '4wires.txt') # grabs the 4
    wire resistance files
num_list = range(len(wire2list)) # makes a range the length
    of the list

wire2numbers = np.zeros((len(wire2list), 4)) # preallocates
    arrays
wire4numbers = np.zeros((len(wire4list), 12))

for i in num_list: # grabs the numbers from the files
    wire2numbers[i] = np.genfromtxt(resist_dir + r'/' +
        wire2list[i])
    wire4numbers[i] = np.genfromtxt(resist_dir + r'/' +
        wire4list[i])

graph_dir = dialog.askdirectory() # directory to put the
    graphs in

# Plots the 4 two wire resistances against trial number

```



```
fig1, ax1 = plt.subplots(1, 1)
wire2plot = ax1.plot(wire2numbers)
ax1.set_xlabel('Trial Number')
ax1.set_ylabel(r'Resistance_($\Omega$)')
fig1.set_size_inches(12, 6)
plt.legend(wire2plot, [r'A$\rightarrow$E', r'B$\rightarrow$F',
    , r'C$\rightarrow$G', r'D$\rightarrow$H'],
    title=r'$I+\rightarrow I-$', loc='lower right')
fig1.savefig(graph_dir + r'/' + 'wire2relaxplot')
```

```
wire4numbers1 = np.delete(wire4numbers, [8, 9, 10, 11], axis
    =1)
wire4numbers2 = np.delete(wire4numbers, [0, 1, 2, 3, 4, 5, 6,
    7], axis=1)
```

*# Plots the last 4 four wire resistances against trial number
(the ones we're interested in)*

```
fig1, ax1 = plt.subplots(1, 1)
wire4plot1 = ax1.plot(wire4numbers1)
ax1.set_xlabel('Trial Number')
ax1.set_ylabel(r'Resistance_($\Omega$)')
fig1.set_size_inches(12, 6)
plt.legend(wire4plot1, [r'A$\rightarrow$E_with_B$\rightarrow$D', r'B$\rightarrow$F_with_C$\rightarrow$E',
    , r'C$\rightarrow$G_with_D$\rightarrow$F', r'D$\rightarrow$H_with_E$\rightarrow$G',
    , r'E$\rightarrow$A_with_F$\rightarrow$H', r'F$\rightarrow$B_with_G$\rightarrow$A',
    , r'G$\rightarrow$C_with_H$\rightarrow$B', r'H$\rightarrow$D_with_A$\rightarrow$C'],
    title=r'$I+\rightarrow I-$_with_V_1+\rightarrow V_1-$', loc='lower right')
fig1.savefig(graph_dir + r'/' + 'wire4relaxplot1')
```

Plots the first 8 four wire resistances against trial number

```
fig2, ax2 = plt.subplots(1, 1)
wire4plot2 = ax2.plot(wire4numbers2)
ax2.set_xlabel('Trial Number')
ax2.set_ylabel(r'Resistance_($\Omega$)')
fig2.set_size_inches(12, 6)
plt.legend(wire4plot2, [r'A$\rightarrow$E_with_C$\rightarrow$G', r'B$\rightarrow$F_with_D$\rightarrow$H',
    , r'C$\rightarrow$G_with_E$\rightarrow$A
```



```

        ', r'D$\rightarrow$H$\rightarrow$F$\rightarrow$B'],
        title=r'$I+\rightarrow I-$ with $V_1+\rightarrow V_1-$', loc='lower right')
fig2.savefig(graph_dir + r'/' + 'wire4relaxplot2')

```

A.2 Pulsing and Measurement - pulseAndMeasure.py

```

import numpy as np
import time
import instruments
import winsound
from tkinter import filedialog as dialog
import matplotlib.pyplot as plt
import matplotlib as mpl
import sys
import os
from tqdm import tqdm

mpl.use("pgf") # sets up mpl to use latex
mpl.rcParams['font.size'] = 15
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "pgf.preamble": [
        r"\usepackage[utf8x]{inputenc}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{cmbright}",
    ]
}

mpl.rcParams.update(style)

pulse1_assignments = {"I+": "B", "I-": "F"} # configuration
    for a pulse from B to F
pulse2_assignments = {"I+": "D", "I-": "H"} # configuration
    for a pulse from D to H
measure_assignments = {"I+": "A", "I-": "E", "V1+": "C", "V1-": "G", "V2+": "B", "V2-": "D"} # here V1 is Vxx
# Swapped BD and CG for to measure vxy on pg

# Fix the saving error and fix go up 5-30mA in 5s then 30+ in
    2s and when signal, 1s. (ended up 5-30 in 5 and then 16
    and then 17-20 in 0.2)

probe_current = 500e-6 # assigns values to the needed

```



```

    constants this is probe current
pulse_current = 19.2e-3 # this is pulsing current
pulse_loop = 2 # this is how many times you want to pulse
    positive and negative
pulse_num = 5 # this code also does single level if there is
    a 1 here
sleep_time = 150e-3 # this is how long between steps it
    sleeps for
sleep_time2 = sleep_time
pulse_width = 1e-3 # the width of the pulse
sep_width = 5 # the time between pulses (not currently used
    in favour of measuring between pulses)
up_down_width = 300 # the time between opposite direction
    pulses (not currently used in favour of measuring this gap
    )
meas_num = 2 # number of times to measure between pulses.
    also governs the pulse separation
pre_pulse_num = 10*5 # number of times to measure just
    before the pulses
pre_pulse_pause = 0.2 # time to wait between pre measurement
    pulses
post_pulse_num = 60*5 # number of times to measure just
    after the pulses
post_pulse_pause = 0.2 # time to wait between post
    measurement pulses

pulse_t = np.zeros((2 * pulse_loop * pulse_num * meas_num))
    # preallocates arrays
probe_t = np.zeros((2 * pulse_loop * pulse_num * meas_num))
vxx = np.zeros((2 * pulse_loop * pulse_num * meas_num))
vxy = np.zeros((2 * pulse_loop * pulse_num * meas_num))
curr = np.zeros((2 * pulse_loop * pulse_num * meas_num))

pre_pulse_vxx = np.zeros((2 * pulse_loop * pre_pulse_num)) #
    preallocates arrays
pre_pulse_vxy = np.zeros((2 * pulse_loop * pre_pulse_num))
pre_pulse_curr = np.zeros((2 * pulse_loop * pre_pulse_num))
pre_probe_t = np.zeros((2 * pulse_loop * pre_pulse_num))

post_pulse_vxx = np.zeros((2 * pulse_loop * post_pulse_num))
    # preallocates arrays
post_pulse_vxy = np.zeros((2 * pulse_loop * post_pulse_num))
post_pulse_curr = np.zeros((2 * pulse_loop * post_pulse_num))
post_probe_t = np.zeros((2 * pulse_loop * post_pulse_num))

sb = instruments.SwitchBox() # sets up the switch box object

```




```
pg = instruments.K2461() # sets up the pulse generator
                             object
mm = instruments.K2000() # sets up the multimeter object

sb.connect(6) # connects to all of the devices
mm.connect(5)
pg.connect()
pbar = tqdm(total=(2 * pulse_loop * (pre_pulse_num +
    post_pulse_num + (pulse_num * meas_num)))) # sets up the
    progress bar

start_t = time.time()
for i in range(pulse_loop): # loops through positive and
    negative switching
    for l in range(pre_pulse_num): # loops through the
        pre pulse measurements
        sb.switch(measure_assignments) # switches to
            the correct assignment
        time.sleep(sleep_time) # sleeps for some
            time to give time to the switchbox to
            switch
        pg.enable_4_wire_probe(probe_current) #
            enables the 4 wire probe
        time.sleep(sleep_time2) # sleeps for some
            time after to let the previous line work
        pg.trigger_fetch() # prepares for
            measurement
        mm.trigger() # prepares for measurement
        pre_probe_t[2 * pre_pulse_num * i + 1] = time
            .time() - start_t # takes the time of the
            measurement
        pre_pulse_vxy[2 * pre_pulse_num * i + 1],
            pre_pulse_curr[2 * pre_pulse_num * i + 1]
            = pg.fetch_one() # fetches the transverse
            voltage and current
        pre_pulse_vxx[2 * pre_pulse_num * i + 1] = mm
            .fetch_one() # fetches the longitudinal
            voltage
        time.sleep(sleep_time2) # sleeps for some
            time after to let the previous line work
        pg.disable_probe_current() # disables the
            probe current
        pbar.update(1) # updates the progress bar
        time.sleep(pre_pulse_pause) # sleeps to give
            some time before next measurement
    for j in range(pulse_num): # loops through the
        number of pulses
```



```

sb.switch(pulse1_assignments)  # switches to
    the correct assignment
time.sleep(sleep_time)  # sleeps for some
    time to give time to the switchbox to
    switch
pg.prepare_pulsing_current(pulse_current,
    pulse_width)  # prepares the pulse
time.sleep(sleep_time)  # sleeps for some
    time after to let the previous line work
pulse_time = time.time() - start_t  # takes
    the time of the pulse
pg.send_pulse()  # sends the pulse
time.sleep(sleep_time)  # sleeps for some
    time after to let the previous line work
sb.switch(measure_assignments)  # switches to
    the correct assignments
time.sleep(sleep_time)  # sleeps for some
    time after to let the previous line work
for k in range(meas_num):
    pg.enable_4_wire_probe(probe_current)
        # enables the 4 wire probe
    time.sleep(sleep_time2)  # sleeps for
        some time after to let the
        previous line work
    pg.trigger_fetch()  # prepares for
        measurement
    mm.trigger()  # prepares for
        measurement
    pulse_t[2 * pulse_num * meas_num * i
        + meas_num * j + k] = pulse_time
        # records the pulse time for these
        measurements
    probe_t[2 * pulse_num * meas_num * i
        + meas_num * j + k] = time.time()
        - start_t  # takes the time of the
        measurement
    vxy[2 * pulse_num * meas_num * i +
        meas_num * j + k], curr[2*
        pulse_num * meas_num * i +
        meas_num * j + k] = pg.fetch_one()
        # fetches the transverse voltage
        and current
    vxx[2 * pulse_num * meas_num * i +
        meas_num * j + k] = mm.fetch_one()
        # fetches the longitudinal
        voltage
    time.sleep(sleep_time2)  # sleeps for

```



```

        some time after to let the
        previous line work
        pg.disable_probe_current() #
        disables the probe current
        pbar.update(1) # updates the
        progress bar
        time.sleep(sleep_time2) # sleeps for
        some time after to let the
        previous line work
    for m in range(post_pulse_num): # loops through the
        post pulse measurements
        sb.switch(measure_assignments) # switches to
        the correct assignment
        time.sleep(sleep_time) # sleeps for some
        time to give time to the switchbox to
        switch
        pg.enable_4_wire_probe(probe_current)
        time.sleep(sleep_time2) # sleeps for some
        time after to let the previous line work
        pg.trigger_fetch() # prepares for
        measurement
        mm.trigger() # prepares for measurement
        post_probe_t[2 * post_pulse_num * i + m] =
            time.time() - start_t # takes the time of
            the measurement
        post_pulse_vxy[2 * post_pulse_num * i + m],
        post_pulse_curr[2 * post_pulse_num * i + m]
        ] = pg.fetch_one() # fetches the
        transverse voltage and current
        post_pulse_vxx[2 * post_pulse_num * i + m] =
            mm.fetch_one() # fetches the longitudinal
            voltage
        time.sleep(sleep_time2) # sleeps for some
        time after to let the previous line work
        pg.disable_probe_current() # disables the
        probe current
        pbar.update(1) # updates the progress bar
        time.sleep(post_pulse_pause) # sleeps to
        give some time before next measurement

# the rest of the loop is the same as the first part,
# but it measures the opposite pulsing
for ll in range(pre_pulse_num, 2 * pre_pulse_num):
    sb.switch(measure_assignments)
    time.sleep(sleep_time)
    pg.enable_4_wire_probe(probe_current)
    time.sleep(sleep_time2)

```



```

pg.trigger_fetch()
mm.trigger()
pre_probe_t[2 * pre_pulse_num * i + 11] =
    time.time() - start_t
pre_pulse_vxy[2 * pre_pulse_num * i + 11],
    pre_pulse_curr[2 * pre_pulse_num * i + 11]
    = pg.fetch_one()
pre_pulse_vxx[2 * pre_pulse_num * i + 11] =
    mm.fetch_one()
time.sleep(sleep_time2)
pg.disable_probe_current()
pbar.update(1)
time.sleep(pre_pulse_pause)
for jj in range(pulse_num, 2*pulse_num):
    sb.switch(pulse2_assignments)
    time.sleep(sleep_time)
    pg.prepare_pulsing_current(pulse_current,
        pulse_width)
    time.sleep(sleep_time)
    pulse_time = time.time() - start_t
    pg.send_pulse()
    time.sleep(sleep_time)
    sb.switch(measure_assignments)
    time.sleep(sleep_time)
    for kk in range(meas_num):
        pg.enable_4_wire_probe(probe_current)
        time.sleep(sleep_time2)
        pg.trigger_fetch()
        mm.trigger()
        pulse_t[2 * pulse_num * meas_num * i
            + meas_num * jj + kk] = pulse_time
        probe_t[2 * pulse_num * meas_num * i
            + meas_num * jj + kk] = time.time
            () - start_t
        vxy[2 * pulse_num * meas_num * i +
            meas_num * jj + kk], curr[2 *
            pulse_num * meas_num * i +
            meas_num * jj + kk] = pg.fetch_one
            ()
        vxx[2 * pulse_num * meas_num * i +
            meas_num * jj + kk] = mm.fetch_one
            ()
        time.sleep(sleep_time2)
        pg.disable_probe_current()
        pbar.update(1)
        time.sleep(sleep_time2)

```



```

        for m2 in range(post_pulse_num, 2 * post_pulse_num):
            sb.switch(measure_assignments)
            time.sleep(sleep_time)
            pg.enable_4_wire_probe(probe_current)
            time.sleep(sleep_time2)
            pg.trigger_fetch()
            mm.trigger()
            post_probe_t[2 * post_pulse_num * i + m2] =
                time.time() - start_t
            post_pulse_vxy[2 * post_pulse_num * i + m2],
            post_pulse_curr[2 * post_pulse_num * i +
            m2] = pg.fetch_one()
            post_pulse_vxx[2 * post_pulse_num * i + m2] =
                mm.fetch_one()
            time.sleep(sleep_time2)
            pg.disable_probe_current()
            pbar.update(1)
            time.sleep(post_pulse_pause)

pg.close()  # closes the connections to the devices
sb.close()
mm.close()

### Saving Data and Graphing
name = dialog.asksaveasfilename(title='Save_data')

# sorts out the names for each of the individual files
name_affix = f'_bfdh-aebdcg'
pre_name_affix = f'_pre_bfdh-aebdcg'
post_name_affix = f'_post_bfdh-aebdcg'
prop_name_affix = '_prop_bfdh-aebdcg'
name_affix = name_affix.replace('.', '-')
pre_name_affix = pre_name_affix.replace('.', '-')
post_name_affix = post_name_affix.replace('.', '-')
name_affix = name_affix + '.txt'
pre_name_affix = pre_name_affix + '.txt'
post_name_affix = post_name_affix + '.txt'
prop_name_affix = prop_name_affix + '.txt'

data = np.hstack((pulse_t, probe_t, vxx, vxy, curr)) #
    stacks the data for saving
pre_data = np.hstack((pre_pulse_vxx, pre_pulse_vxy,
    pre_pulse_curr, pre_probe_t))
post_data = np.hstack((post_pulse_vxx, post_pulse_vxy,
    post_pulse_curr, post_probe_t))
prop_data = np.hstack((probe_current, pulse_current,
    pulse_loop, pulse_num, sleep_time, sleep_time2,

```



```

pulse_width, sep_width, up_down_width, meas_num,
pre_pulse_num, pre_pulse_pause, post_pulse_num,
post_pulse_pause))

if name:  # does the auto naming so it is pain free to save
it
    name = name.replace('_after_2wires', '')
    name = name.replace('_after_4wires', '')
    name = name.replace('_before_2wires', '')
    name = name.replace('_before_4wires', '')
    name_data = name.replace('.txt', name_affix)
    name_pre_data = name.replace('.txt', pre_name_affix)
    name_post_data = name.replace('.txt', post_name_affix
    )
    name_prop_data = name.replace('.txt', prop_name_affix
    )
    np.savetxt(name_data, data, newline='\n', delimiter='
\t')  # save
    np.savetxt(name_pre_data, pre_data, newline='\n',
        delimiter='\t')
    np.savetxt(name_post_data, post_data, newline='\n',
        delimiter='\t')
    np.savetxt(name_prop_data, prop_data, newline='\n',
        delimiter='\t')
    print(f'Data saved as {name_data}')
else:
    print('No Filename: Data not saved')

# plots Rxy across whole range (for checking if it's working,
    actual graphs are from singlelevelGraph.py and
    multilevelGraph.py)
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(probe_t, vxy/curr, 'kx')
res_main_plot2 = ax1.plot(probe_t, vxy/curr, 'k-', linewidth
    =0.5)
res_main_plot3 = ax1.plot(pre_probe_t, pre_pulse_vxy/
    pre_pulse_curr, 'gx')
res_main_plot4 = ax1.plot(post_probe_t, post_pulse_vxy/
    post_pulse_curr, 'rx')
for a in pulse_t:
    plt.axvline(x=a, lw=0.1, color='b')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel(r'Resistance$_{xy}$ ($\Omega$)')
fig1.set_size_inches(12, 6)

# plots Rxx across whole range
fig2, ax2 = plt.subplots(1, 1)

```



```

res_main_plot2a = ax2.plot(probe_t, vxx/curr, 'kx')
res_main_plot3a = ax2.plot(pre_probe_t, pre_pulse_vxx/
    pre_pulse_curr, 'gx')
res_main_plot4a = ax2.plot(post_probe_t, post_pulse_vxx/
    post_pulse_curr, 'rx')
ax2.set_xlabel('Time(s)')
ax2.set_ylabel(r'Resistance$_{\text{xx}}$($\Omega$)')
fig2.set_size_inches(12, 6)

# Saves the graphs
name_graph = name.replace('.txt', '')
name_graph_affix = name_affix.replace('.txt', '_')
fig1.savefig(name_graph + name_graph_affix + 'plotxy' + '.pdf')
fig2.savefig(name_graph + name_graph_affix + 'plotxx' + '.pdf')

```

A.3 Graphing

A.3.1 Single-level Graphing - singlelevelGraph.py

```

import numpy as np
from tkinter import filedialog as dialog
import matplotlib.pyplot as plt
import matplotlib as mpl
import re

mpl.use("pgf") # sets up mpl to use latex
mpl.rcParams['font.size'] = 18
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "pgf.preamble": [
        r"\usepackage[utf8x]{inputenc}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{cmbright}",
    ]
}
mpl.rcParams.update(style)

%% Data is of the format pulse_t probe_t vxx vxy curr, and
    has the title bfdh-aebdcg-pc-probe_current_
# puc-pulse_current_pl-pulse_loop_pn-pulse_num_pw-
    pulse_width_sw-sep_width

name = dialog.askopenfilename(title="Open Data file") #

```



```

    select data file to plot
pre_name = name.replace('_bfdh', '_pre_bfdh') # sorts out
    the names of the other files to load
post_name = name.replace('_bfdh', '_post_bfdh')

data = np.genfromtxt(name) # loads up the file data
pre_data = np.genfromtxt(pre_name)
post_data = np.genfromtxt(post_name)

data_num = 5 # number of sections of data in that file
pre_num = 4
post_num = 4

pulse_t = data[0:int(len(data)/data_num)] # grabs the
    necessary data from the files
probe_t = data[int(len(data)/data_num):int(2*len(data)/
    data_num)]
vxx = data[int(2*len(data)/data_num):int(3*len(data)/data_num
    )]
vxy = data[int(3*len(data)/data_num):int(4*len(data)/data_num
    )]
curr = data[int(4*len(data)/data_num):]

pre_pulse_vxx = pre_data[0:int(len(pre_data)/pre_num)] #
    grabs the necessary data from the files
pre_pulse_vxy = pre_data[int(len(pre_data)/pre_num):int(2*len
    (pre_data)/pre_num)]
pre_pulse_curr = pre_data[int(2*len(pre_data)/pre_num):int(3*
    len(pre_data)/pre_num)]
pre_probe_t = pre_data[int(3*len(pre_data)/pre_num):]

post_pulse_vxx = post_data[0:int(len(post_data)/post_num)] #
    grabs the necessary data from the files
post_pulse_vxy = post_data[int(len(post_data)/post_num):int
    (2*len(post_data)/post_num)]
post_pulse_curr = post_data[int(2*len(post_data)/post_num):
    int(3*len(post_data)/post_num)]
post_probe_t = post_data[int(3*len(post_data)/post_num):]

pulse_current_str = re.findall(r'_puc-0-\d+', name) # as the
    single level switching is in a different format to what
    it would now produce, this grabs the necessary parameters
pulse_current_str = pulse_current_str[0]
pulse_current_str = pulse_current_str.replace('-', '.')
pulse_current = float(pulse_current_str[5:])

```




```

post_pulse_num_str = re.findall(r'_popn-\d+', name) # as the
    single level switching is in a different format to what
    it would now produce, this grabs the necessary parameters
post_pulse_num_str = post_pulse_num_str[0]
post_pulse_num_str = post_pulse_num_str.replace('-', '.')
post_pulse_num = int(post_pulse_num_str[6:])

pre_pulse_num_str = re.findall(r'_ppn-\d+', name) # as the
    single level switching is in a different format to what it
    would now produce, this grabs the necessary parameters
pre_pulse_num_str = pre_pulse_num_str[0]
pre_pulse_num_str = pre_pulse_num_str.replace('-', '.')
pre_pulse_num = int(pre_pulse_num_str[5:])

pulse_loop_str = re.findall(r'_pl-\d+', name) # as the
    single level switching is in a different format to what it
    would now produce, this grabs the necessary parameters
pulse_loop_str = pulse_loop_str[0]
pulse_loop_str = pulse_loop_str.replace('-', '.')
pulse_loop = int(pulse_loop_str[4:])

post_pulse_set_tim = np.zeros(2 * pulse_loop) # preallocates
    for arrays
pre_pulse_set_tim = np.zeros(2 * pulse_loop)

# finds the average time that the pre pulse measurements run
    for
for i in range(2 * pulse_loop-1):
    pre_pulse_set_tim[i] = pre_probe_t[pre_pulse_num*(i+1)-1]
        - pre_probe_t[pre_pulse_num*i]
pre_pulse_tim = np.mean(pre_pulse_set_tim[:-1])

# finds the average time that the post pulse measurements run
    for
for i in range(2 * pulse_loop-1):
    post_pulse_set_tim[i] = post_probe_t[post_pulse_num*(i+1)
        -1] - post_probe_t[post_pulse_num*i]
post_pulse_tim = np.mean(post_pulse_set_tim[:-1])

# plots Rxx against timings
fig2, ax2 = plt.subplots(1, 1)
res_main_plot2a = ax2.plot(probe_t, vxx/curr, 'kx')
res_main_plot3a = ax2.plot(pre_probe_t, pre_pulse_vxx/
    pre_pulse_curr, 'gx')
res_main_plot4a = ax2.plot(post_probe_t, post_pulse_vxx/
    post_pulse_curr, 'rx')

```



```

ax2.set_xlabel('Time (s)')
ax2.set_ylabel(r'Resistance$_{xx}$ ($\Omega$)')
fig2.set_size_inches(12, 9)
name_graph = name.replace('.txt', '')
fig2.savefig(name_graph + '_plotxx_ad' + '.pdf') # saves the
graph

# plots Rxy against timings
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(probe_t, vxy/curr, 'kx-', linewidth
    =0.5, label=f'Measurements of Pulses')
res_main_plot3 = ax1.plot(pre_probe_t, pre_pulse_vxy/
    pre_pulse_curr, 'gx', label=f'Measurements prior to the
    set of pulses,' + '\n' + f'lasting around {np.round(
    pre_pulse_tim,2)}s')
res_main_plot4 = ax1.plot(post_probe_t, post_pulse_vxy/
    post_pulse_curr, 'rx', label=f'Measurements after the set
    of pulses,' + '\n' + f'lasting around {np.round(
    post_pulse_tim,2)}s')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel(r'Resistance$_{xy}$ ($\Omega$)')
fig1.set_size_inches(12, 9)
plt.legend(title=f'Pulses were at {pulse_current} A', loc='
    lower right')
fig1.savefig(name_graph + '_plotxy' + '.pdf') # saves the
graph

```

A.3.2 Multilevel Graphing - multilevelGraph.py

```

import numpy as np
from tkinter import filedialog as dialog
import matplotlib.pyplot as plt
import matplotlib as mpl

mpl.use("pgf") # sets up mpl to use latex
mpl.rcParams['font.size'] = 18
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "pgf.preamble": [
        r"\usepackage[utf8x]{inputenc}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{cmbright}",
    ]
}
mpl.rcParams.update(style)

```



```

name = dialog.askopenfilename(title="Open Data file") #
    select data file to plot
pre_name = name.replace('_bfdh', '_pre_bfdh') # sorts out
    the names of the other files to load
post_name = name.replace('_bfdh', '_post_bfdh')
prop_name = name.replace('_bfdh', '_prop_bfdh')

data = np.genfromtxt(name) # loads up the file data
pre_data = np.genfromtxt(pre_name)
post_data = np.genfromtxt(post_name)
prop_data = np.genfromtxt(prop_name)

data_num = 5 # number of sections of data in that file
pre_num = 4
post_num = 4

pulse_t = data[0:int(len(data)/data_num)] # grabs the
    necessary data from the files
probe_t = data[int(len(data)/data_num):int(2*len(data)/
    data_num)]
vxx = data[int(2*len(data)/data_num):int(3*len(data)/data_num
    )]
vxy = data[int(3*len(data)/data_num):int(4*len(data)/data_num
    )]
curr = data[int(4*len(data)/data_num):]

pre_pulse_vxx = pre_data[0:int(len(pre_data)/pre_num)] #
    grabs the necessary data from the files
pre_pulse_vxy = pre_data[int(len(pre_data)/pre_num):int(2*len
    (pre_data)/pre_num)]
pre_pulse_curr = pre_data[int(2*len(pre_data)/pre_num):int(3*
    len(pre_data)/pre_num)]
pre_probe_t = pre_data[int(3*len(pre_data)/pre_num):]

post_pulse_vxx = post_data[0:int(len(post_data)/post_num)] #
    grabs the necessary data from the files
post_pulse_vxy = post_data[int(len(post_data)/post_num):int
    (2*len(post_data)/post_num)]
post_pulse_curr = post_data[int(2*len(post_data)/post_num):
    int(3*len(post_data)/post_num)]
post_probe_t = post_data[int(3*len(post_data)/post_num):]

probe_current = prop_data[0] # grabs the necessary property
    values from the files
pulse_current = prop_data[1]
pulse_loop = int(prop_data[2])
pulse_num = int(prop_data[3])

```



```

sleep_time = prop_data[4]
sleep_time2 = prop_data[5]
pulse_width = prop_data[6]
sep_width = prop_data[7]
up_down_width = prop_data[8]
meas_num = int(prop_data[9])
pre_pulse_num = int(prop_data[10])
pre_pulse_pause = prop_data[11]
post_pulse_num = int(prop_data[12])
post_pulse_pause = prop_data[13]

pulse_sep = np.diff(pulse_t)  # calculates the time gaps
                               # between successive pulses
pulse_sep = pulse_sep[pulse_sep != 0]  # gets rid of the ones
                                       # that are zero

oppo_time_set = np.zeros(2 * pulse_loop - 1)  # preallocates
                                                # for arrays

for i in range(0, 2 * pulse_loop-1):  # finds the average
    time between opposite direction pulses
    oppo_time_set[i] = pulse_sep[pulse_num * (i + 1) - 1]
opposite_gap = np.mean(oppo_time_set)

same_time_set = np.zeros(2 * (pulse_num - 1) * pulse_loop)  #
                                                                # preallocates for arrays

for i in range(2*pulse_loop):  # finds the average time
    between same direction pulses
    same_time_set[i * (pulse_num - 1):(i + 1) * (pulse_num -
        1)] = pulse_sep[i * pulse_num:(i + 1) * pulse_num - 1]
same_gap = np.mean(same_time_set)

post_pulse_set_tim = np.zeros(2 * pulse_loop)  # preallocates
                                                # for arrays
pre_pulse_set_tim = np.zeros(2 * pulse_loop)

# finds the average time that the pre pulse measurements run
for
for i in range(2 * pulse_loop-1):
    pre_pulse_set_tim[i] = pre_probe_t[pre_pulse_num*(i+1)-1]
    - pre_probe_t[pre_pulse_num*i]
pre_pulse_tim = np.mean(pre_pulse_set_tim[:-1])

# finds the average time that the post pulse measurements run
for
for i in range(2 * pulse_loop-1):

```



```

        post_pulse_set_tim[i] = post_probe_t[post_pulse_num*(i+1)
        -1] - post_probe_t[post_pulse_num*i]
post_pulse_tim = np.mean(post_pulse_set_tim[:-1])

# plots Rxy against timings, for a specific range of time so
it is easier to see
fig1, ax1 = plt.subplots(1, 1)
fff = ax1.plot(probe_t[:,meas_num], vxy[:,meas_num]/curr[:,
    meas_num], 'kx', label=f'Initial Measurement of Pulse')
res_main_plot = ax1.plot(probe_t, vxy/curr, 'k-', linewidth
    =0.5, label=f'Measurements Between Pulses, {meas_num} Points Between Each')
res_main_plot3 = ax1.plot(pre_probe_t, pre_pulse_vxy/
    pre_pulse_curr, 'gx', label=f'Measurements prior to the set of pulses, ' + '\n' + f'lasting around {np.round(
    pre_pulse_tim,2)}s')
res_main_plot4 = ax1.plot(post_probe_t, post_pulse_vxy/
    post_pulse_curr, 'rx', label=f'Measurements after the set of pulses, ' + '\n' + f'lasting around {np.round(
    post_pulse_tim,2)}s')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel(r'Resistance$_{xy}$$_{(\Omega)}$')
fig1.set_size_inches(12, 9)
ax1.set_xlim((0, 500))
ax1.set_ylim((0.165, 0.19))
plt.legend(title=f'Pulses were at {pulse_current} A, were a mean of {np.round(same_gap,2)}s' + '\n' + f'separation between same direction pulses, and {np.round(opposite_gap,2)}s' + '\n' + f'between opposite pulses', loc='lower right')

# plots Rxx against timings
fig2, ax2 = plt.subplots(1, 1)
res_main_plot2a = ax2.plot(probe_t, vxx/curr, 'kx')
res_main_plot3a = ax2.plot(pre_probe_t, pre_pulse_vxx/
    pre_pulse_curr, 'gx')
res_main_plot4a = ax2.plot(post_probe_t, post_pulse_vxx/
    post_pulse_curr, 'rx')
ax2.set_xlabel('Time (s)')
ax2.set_ylabel(r'Resistance$_{xx}$$_{(\Omega)}$')
fig2.set_size_inches(12, 9)
name_graph = name.replace('.txt', '')
fig1.savefig(name_graph + '_plotxy_ad' + '.pdf') # saves the graph
fig2.savefig(name_graph + '_plotxx_ad' + '.pdf') # saves the graph

```



```

# plots Rxy against timings
fig1, ax1 = plt.subplots(1, 1)
fff = ax1.plot(probe_t[:, meas_num], vxy[:, meas_num]/curr[:,
    meas_num], 'kx', label=f'Initial Measurement of Pulse')
res_main_plot = ax1.plot(probe_t, vxy/curr, 'k-', linewidth
    =0.5, label=f'Measurements Between Pulses, {meas_num}
    Points Between Each')
res_main_plot3 = ax1.plot(pre_probe_t, pre_pulse_vxy/
    pre_pulse_curr, 'gx', label=f'Measurements prior to the
    set of pulses,' + '\n' + f'lasting around {np.round(
    pre_pulse_tim, 2)}s')
res_main_plot4 = ax1.plot(post_probe_t, post_pulse_vxy/
    post_pulse_curr, 'rx', label=f'Measurements after the set
    of pulses,' + '\n' + f'lasting around {np.round(
    post_pulse_tim, 2)}s')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel(r'Resistance$_{xy}$ ($\Omega$)')
fig1.set_size_inches(12, 9)
ax1.set_ylim((0.165, 0.19))
plt.legend(title=f'Pulses were at {pulse_current} A, were a
    mean of {np.round(same_gap, 2)}s' + '\n' + f'separation
    between same direction pulses, and {np.round(opposite_gap,
    2)}s' + '\n' + f'between opposite pulses', loc='lower
    right')
fig1.savefig(name_graph + '_plotxy' + '.pdf') # saves the
graph

```

A.4 Analysis

A.4.1 Single Level Decay Analysis - decayAnalysis.py

```

import numpy as np
import re
from tkinter import filedialog as dialog
import matplotlib.pyplot as plt
import matplotlib as mpl
import os
from tqdm import tqdm
import scipy.optimize as spom

mpl.use("pgf") # sets up mpl to use latex
mpl.rcParams['font.size'] = 18
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "pgf.preamble": [
        r"\usepackage[utf8x]{inputenc}",

```



```

        r"\usepackage[T1]{fontenc}",
        r"\usepackage{cmbright}",
    ]
}
mpl.rcParams.update(style)

def kohl_exp(t, a, b, d): # defines the Kohlrausch
    Exponential that we've chosen to fit to
    order = -(t/b)**(0.6))
    rxy = a * np.exp(order) + d
    return rxy

file_list = dialog.askopenfilenames(title="Open Data File")
    # open the data files you want to analyse (the ones with
    post in)
file_list_length = len(file_list) # finds the length of the
    file list

data_num = 5 # number of sections of data in that file
pre_num = 4
post_num = 4

pulse_current_str = re.findall(r'_puc-0-\d+', file_list[0])
    # as the single level switching is in a different format
    to what it would now produce, this grabs the necessary
    parameters
pulse_current_str = pulse_current_str[0]
pulse_current_str = pulse_current_str.replace('-', '.')
pulse_current = float(pulse_current_str[5:])

post_pulse_num_str = re.findall(r'_popn-\d+', file_list[0])
    # as the single level switching is in a different format
    to what it would now produce, this grabs the necessary
    parameters
post_pulse_num_str = post_pulse_num_str[0]
post_pulse_num_str = post_pulse_num_str.replace('-', '.')
post_pulse_num = int(post_pulse_num_str[6:])

post_data = np.genfromtxt(file_list[0]) # loads up the file
    data

post_pulse_vxx = post_data[0:int(len(post_data)/post_num)] #
    grabs the necessary file data
post_pulse_vxy = post_data[int(len(post_data)/post_num):int
    (2*len(post_data)/post_num)]
post_pulse_curr = post_data[int(2*len(post_data)/post_num):

```



```

    int(3*len(post_data)/post_num)]
post_probe_t = post_data[int(3*len(post_data)/post_num):]

count = post_pulse_num # calculates necessary parameters to
    preallocate arrays
num = np.shape(post_data)
num = num[0]/post_num
run = int(num/count)
pos_tim = np.zeros((count, run, file_list_length)) #
    preallocates for arrays
pos_rxy = np.zeros((count, run, file_list_length))

result = np.zeros((file_list_length, run, 3)) # preallocates
    for arrays
resvar = np.zeros((file_list_length, run, 3))
respuc = np.zeros((file_list_length))

for i in tqdm(range(file_list_length)): # loops through all
    the files for all the pulse currents
    post_data = np.genfromtxt(file_list[i]) # loads up the
        file data
    post_pulse_vxx = post_data[0:int(len(post_data)/post_num)
        ] # grabs the necessary file data
    post_pulse_vxy = post_data[int(len(post_data)/post_num):
        int(2*len(post_data)/post_num)]
    post_pulse_curr = post_data[int(2*len(post_data)/post_num
        ):int(3*len(post_data)/post_num)]
    post_probe_t = post_data[int(3*len(post_data)/post_num):]

    pulse_current_str = re.findall(r'_puc-0-\d+', file_list[i]
        ]) # as the single level switching is in a different
        format to what it would now produce, this grabs the
        necessary parameters
    pulse_current_str = pulse_current_str[0]
    pulse_current_str = pulse_current_str.replace('-', '.')
    pulse_current = float(pulse_current_str[5:])
    respuc[i] = pulse_current

    post_pulse_rxy = post_pulse_vxy / post_pulse_curr #
        calculates the Rxy
    offset = np.mean(post_pulse_rxy)
    scale = (np.max(post_pulse_rxy)-offset)
    run = int(len(post_pulse_vxy)/post_pulse_num)

    temp_analysi_matr = np.zeros((post_pulse_num, run)) #
        preallocates for arrays
    temp_time_matr = np.zeros((post_pulse_num, run))

```




```

scale = np.zeros((run))
offset = np.zeros((run))
for j in range(run):
    if j % 2 == 0:
        temp_analysi_matr[:, j] = post_pulse_rxy[(j *
            post_pulse_num):((j + 1) * post_pulse_num)] #
            assigns data to new matrix
        temp_time_matr[:, j] = post_probe_t[(j *
            post_pulse_num):((j + 1) * post_pulse_num)] -
            np.min(post_probe_t[(j * post_pulse_num):((j +
            1) * post_pulse_num)]) # assigns data to new
            matrix
        offset[j] = np.mean(post_pulse_rxy) # gets
            initial starting point for curve fitting for
            the offset
        scale[j] = 2 * (np.min(temp_analysi_matr[:, j]) -
            np.mean(temp_analysi_matr[:, j])) # gets
            initial starting point for curve fitting for
            the scale

    else:
        temp_analysi_matr[:, j] = post_pulse_rxy[(j *
            post_pulse_num):((j + 1) * post_pulse_num)] #
            assigns data to new matrix
        temp_time_matr[:, j] = post_probe_t[(j *
            post_pulse_num):((j + 1) * post_pulse_num)] -
            np.min(post_probe_t[(j * post_pulse_num):((j +
            1) * post_pulse_num)]) # assigns data to new
            matrix
        offset[j] = np.mean(post_pulse_rxy) # gets
            initial starting point for curve fitting for
            the offset
        scale[j] = 2 * (np.max(temp_analysi_matr[:, j]) -
            np.mean(temp_analysi_matr[:, j])) # gets
            initial starting point for curve fitting for
            the scale

    val, var = spom.curve_fit(kohl_exp, temp_time_matr[:,
        j], temp_analysi_matr[:, j], p0=[scale[j], 1,
        offset[j]], maxfev=5000000, method='lm') # curve
        fits the data
    result[i, j, :] = val # stores the parameters
        generated
    resvar[i, j, :] = np.sqrt(np.diag(var)) # calculates
        and stores the 1 sigma errors on the parameters
    fig1, ax1 = plt.subplots(1, 1) # plots the fit for
        all of the data

```



```

ax1.plot(temp_time_matr[:, j], temp_analysi_matr[:, j]
        ], 'kx', label='Data Obtained from Experiment')
ax1.plot(temp_time_matr[:, j], kohl_exp(
        temp_time_matr[:, j], val[0], val[1], val[2]), 'k-
        ', label=r'Line of Best Fit in the Form  $R_{xy} = a \cdot \exp\left[-\left(\frac{t}{b}\right)^{0.6}\right] + c$ ')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel(r'Resistance  $R_{xy}(\Omega)$ ')
plt.legend(title=f'Pulse Current of {pulse_current} mA')
fig1.set_size_inches(12, 9)
fig1.savefig(os.getcwd() + r'\\' + f'test{i}{j}.pdf')
        # saves the graph

```

```

resvar[resvar == np.inf] = np.NaN # gets rid of the infinite
    errors
result[resvar == np.inf] = np.NaN # gets rid of the infinite
    errors' parameters
variation = resvar/np.abs(result) # finds the relative error
sqvariation = variation**2 # squares the relative error
sqvariation[sqvariation > 100000] = np.NaN # gets rid of
    extremely high numbers (erroneous fits)
value = np.nanmean(sqvariation, axis=(1, 2)) # calculates
    the mean disregarding NaN values
sprtvalu = np.sqrt(value) # square roots those values to get
    a "fitness value"

```

```

fig2, ax2 = plt.subplots(1, 1) # initialises the graph
res_main_plot2a = ax2.plot(respuc, sprtvalu, 'kx-') # plots
    the fitness value against the pulse current
ax2.set_xlabel('Pulse Current (mA)') # labels the x-axis
ax2.set_ylabel(r'Fitness Value (Arbitrary Units)') # labels
    the y-axis
fig2.set_size_inches(12, 9) # sets the size of the graph
fig2.savefig('plotdecayanalysis21.pdf') # saves the graph

```

A.4.2 Multilevel Gap Analysis: Pulse Current - splitPucAnalysis.py

```

import numpy as np
from tkinter import filedialog as dialog
import matplotlib.pyplot as plt
import matplotlib as mpl

mpl.use("pgf") # sets up mpl to use latex

```



```

mpl.rcParams['font.size'] = 18
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "pgf.preamble": [
        r"\usepackage[utf8x]{inputenc}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{cmbright}",
    ]
}
mpl.rcParams.update(style)

file_list = dialog.askopenfilenames(title="Open Data Files")
    # open the data files you want to analyse
pre_list = [f.replace('_bfdh', '_pre_bfdh') for f in
    file_list] # sorts out the names of the other files to
    load
post_list = [f.replace('_bfdh', '_post_bfdh') for f in
    file_list]
prop_list = [f.replace('_bfdh', '_prop_bfdh') for f in
    file_list]

full_pos_level = np.array([], dtype=np.int64).reshape(0, 4)
    # sets up arrays to concatenate into
full_pos_time = np.array([], dtype=np.int64).reshape(0, 4)
full_pos_puc = np.array([], dtype=np.int64).reshape(0, 4)
full_neg_level = np.array([], dtype=np.int64).reshape(0, 4)
full_neg_time = np.array([], dtype=np.int64).reshape(0, 4)
full_neg_puc = np.array([], dtype=np.int64).reshape(0, 4)

for i in range(len(file_list)): # loops through all the
    files for all the pulse currents

    data = np.genfromtxt(file_list[i]) # loads up the file
        data
    pre_data = np.genfromtxt(pre_list[i])
    post_data = np.genfromtxt(post_list[i])
    prop_data = np.genfromtxt(prop_list[i])

    data_num = 5 # number of sections of data in that file
    pre_num = 4
    post_num = 4

    pulse_t = data[0:int(len(data)/data_num)] # grabs the
        necessary data from the files
    probe_t = data[int(len(data)/data_num):int(2*len(data)/

```



```

    data_num)]
vxx = data[int(2*len(data)/data_num):int(3*len(data)/
    data_num)]
vxy = data[int(3*len(data)/data_num):int(4*len(data)/
    data_num)]
curr = data[int(4*len(data)/data_num):]

pre_pulse_vxx = pre_data[0:int(len(pre_data)/pre_num)] #
    grabs the necessary data from the files
pre_pulse_vxy = pre_data[int(len(pre_data)/pre_num):int
    (2*len(pre_data)/pre_num)]
pre_pulse_curr = pre_data[int(2*len(pre_data)/pre_num):
    int(3*len(pre_data)/pre_num)]
pre_probe_t = pre_data[int(3*len(pre_data)/pre_num):]

post_pulse_vxx = post_data[0:int(len(post_data)/post_num)
    ] # grabs the necessary data from the files
post_pulse_vxy = post_data[int(len(post_data)/post_num):
    int(2*len(post_data)/post_num)]
post_pulse_curr = post_data[int(2*len(post_data)/post_num
    ):int(3*len(post_data)/post_num)]
post_probe_t = post_data[int(3*len(post_data)/post_num):]

probe_current = prop_data[0] # grabs the necessary
    property values from the files
pulse_current = prop_data[1]
pulse_loop = int(prop_data[2])
pulse_num = int(prop_data[3])
sleep_time = prop_data[4]
sleep_time2 = prop_data[5]
pulse_width = prop_data[6]
sep_width = prop_data[7]
up_down_width = prop_data[8]
meas_num = int(prop_data[9])
pre_pulse_num = int(prop_data[10])
pre_pulse_pause = prop_data[11]
post_pulse_num = int(prop_data[12])
post_pulse_pause = prop_data[13]

pulse_sep = np.diff(pulse_t) # calculates the time gaps
    between successive pulses
pulse_sep = pulse_sep[pulse_sep != 0] # gets rid of the
    ones that are zero
rxy = vxy/curr # calculates rxy
rxy_pulse = rxy[1::meas_num] # picks the value of the
    resistance from the second measurement in every pulse
level_sep = np.diff(rxy_pulse) # calculates the

```



```

    difference

    same_time_set = np.zeros((2 * pulse_loop, pulse_num-1))
    # preallocates for arrays
    same_level_set = np.zeros((2 * pulse_loop, pulse_num-1))

    for i in range(2*pulse_loop):
        same_time_set[i, :] = pulse_sep[i * pulse_num:(i + 1)
            * pulse_num - 1] # calculates the time
            separation
        same_level_set[i, :] = level_sep[i * pulse_num:(i +
            1) * pulse_num - 1] # calculates the level
            separation
    pos_level_sep = same_level_set[:, :2, :] # grabs the
        positive level separations
    pos_time_sep = same_time_set[:, :2, :]
    neg_level_sep = same_level_set[:, 1:2, :] # grabs the
        negative level separations
    neg_time_sep = same_time_set[:, 1:2, :]

    # structures arrays for the pulse current
    pos_puc = np.array([pulse_current, pulse_current,
        pulse_current, pulse_current],
        [pulse_current, pulse_current,
            pulse_current, pulse_current])
    neg_puc = np.array([pulse_current, pulse_current,
        pulse_current, pulse_current],
        [pulse_current, pulse_current,
            pulse_current, pulse_current])

    # concatenates arrays together to record the values found
    full_pos_level = np.concatenate((full_pos_level,
        pos_level_sep), 0)
    full_pos_time = np.concatenate((full_pos_time,
        pos_time_sep), 0)
    full_pos_puc = np.concatenate((full_pos_puc, pos_puc), 0)
    full_neg_level = np.concatenate((full_neg_level,
        neg_level_sep), 0)
    full_neg_time = np.concatenate((full_neg_time,
        neg_time_sep), 0)
    full_neg_puc = np.concatenate((full_neg_puc, neg_puc), 0)

    mean_neg_level = np.mean(np.array([full_neg_level[:, :2, :],
        full_neg_level[:, 1:2, :]]), axis=0) # finds the mean value
        for each file/pulse current
    mean_neg_time = np.mean(np.array([full_neg_time[:, :2, :],

```



```

    full_neg_time[1::2, :])) , axis=0)
mean_neg_puc = np.mean(np.array([full_neg_puc[:, 2, :],
    full_neg_puc[1::2, :]]), axis=0)
mean_pos_level = np.mean(np.array([full_pos_level[:, 2, :],
    full_pos_level[1::2, :]]), axis=0)
mean_pos_time = np.mean(np.array([full_pos_time[:, 2, :],
    full_pos_time[1::2, :]]), axis=0)
mean_pos_puc = np.mean(np.array([full_pos_puc[:, 2, :],
    full_pos_puc[1::2, :]]), axis=0)

# plots the pulse separation against the 1st negative pulse's
# pulse current
full_neg_time_mean_1 = np.mean(np.array([full_neg_time[:, 2,
    0], full_neg_time[:, 2, 1], full_neg_time[:, 2, 2],
    full_neg_time[:, 2, 3]]))
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(1000*full_neg_puc[:, 2, 0],
    full_neg_level[:, 2, 0], 'kx-', label=r'Level_1$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*full_neg_puc[:, 2, 1],
    full_neg_level[:, 2, 1], 'rx-', label=r'Level_2$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*full_neg_puc[:, 2, 2],
    full_neg_level[:, 2, 2], 'bx-', label=r'Level_3$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*full_neg_puc[:, 2, 3],
    full_neg_level[:, 2, 3], 'gx-', label=r'Level_4$\rightarrow$Resistance Separation')
ax1.set_xlabel(r'Pulse Current (mA)')
ax1.set_ylabel(r'Difference in Level Resistance$_{xy}$ ($\Omega$)')
plt.legend(title=r'Positive Pulses, 1st Set,' + '\n' + f'Mean
    Pulse Separation of {np.round(full_neg_time_mean_1, 2)}s'
    )
fig1.set_size_inches(12, 6)
fig1.savefig('puc_pos1' + '.pdf')
plt.show()

# plots the pulse separation against the 2nd negative pulse's
# pulse current
full_neg_time_mean_2 = np.mean(np.array([full_neg_time[1::2,
    0], full_neg_time[1::2, 1], full_neg_time[1::2, 2],
    full_neg_time[1::2, 3]]))
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(1000*full_neg_puc[1::2, 0],
    full_neg_level[1::2, 0], 'kx-', label=r'Level_1$\rightarrow$Resistance Separation')

```



```

res_main_plot = ax1.plot(1000*full_neg_puc[1::2, 1],
    full_neg_level[1::2, 1], 'rx-', label=r'Level_2$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*full_neg_puc[1::2, 2],
    full_neg_level[1::2, 2], 'bx-', label=r'Level_3$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*full_neg_puc[1::2, 3],
    full_neg_level[1::2, 3], 'gx-', label=r'Level_4$\rightarrow$Resistance Separation')
ax1.set_xlabel(r'Pulse Current (mA)')
ax1.set_ylabel(r'Difference in Level Resistance$_{xy}$ ($\Omega$)')
plt.legend(title=r'Positive Pulses, 2nd Set,' + '\n' + f'Mean Pulse Separation of {np.round(full_neg_time_mean_2, 2)}s')
fig1.set_size_inches(12, 6)
fig1.savefig('puc_pos2' + '.pdf')
plt.show()

# plots the pulse separation against the 1st positive pulse's pulse current
full_pos_time_mean_1 = np.mean(np.array([full_pos_time[:, 0], full_pos_time[:, 1], full_pos_time[:, 2], full_pos_time[:, 3]]))
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 0],
    mean_pos_level[:, 0], 'kx-', label=r'Level_1$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 1],
    mean_pos_level[:, 1], 'rx-', label=r'Level_2$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 2],
    mean_pos_level[:, 2], 'bx-', label=r'Level_3$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 3],
    mean_pos_level[:, 3], 'gx-', label=r'Level_4$\rightarrow$Resistance Separation')
ax1.set_xlabel(r'Pulse Current (mA)')
ax1.set_ylabel(r'Difference in Level Resistance$_{xy}$ ($\Omega$)')
plt.legend(title=r'Negative Pulses, 1st Set,' + '\n' + f'Mean Pulse Separation of {np.round(full_neg_time_mean_1, 2)}s')
fig1.set_size_inches(12, 6)
fig1.savefig('puc_neg1' + '.pdf')
plt.show()

```



```

# plots the pulse separation against the 2nd positive pulse's
  pulse current
full_pos_time_mean_2 = np.mean(np.array([full_pos_time[1::2,
0], full_pos_time[1::2, 1], full_pos_time[1::2, 2],
full_pos_time[1::2, 3]]))
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(1000*mean_pos_puc[1::2, 0],
mean_pos_level[1::2, 0], 'kx-', label=r'Level_1$\rightarrow$
Resistance Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[1::2, 1],
mean_pos_level[1::2, 1], 'rx-', label=r'Level_2$\rightarrow$
Resistance Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[1::2, 2],
mean_pos_level[1::2, 2], 'bx-', label=r'Level_3$\rightarrow$
Resistance Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[1::2, 3],
mean_pos_level[1::2, 3], 'gx-', label=r'Level_4$\rightarrow$
Resistance Separation')
ax1.set_xlabel(r'Pulse Current (mA)')
ax1.set_ylabel(r'Difference in Level Resistance$_{xy}$ ($\Omega$)')
plt.legend(title=r'Negative Pulses, 2nd Set,' + '\n' + f'Mean
Pulse Separation of {np.round(full_neg_time_mean_2, 2)}s'
)
fig1.set_size_inches(12, 6)
fig1.savefig('puc_neg2' + '.pdf')
plt.show()

# plots the pulse separation against the mean negative pulse's
  pulse current
full_neg_time_mean_mean = np.mean(np.array([
full_neg_time_mean_1, full_neg_time_mean_2]))
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(1000*mean_neg_puc[:, 0], -
mean_neg_level[:, 0], 'kx-', label=r'Level_1$\rightarrow$
Resistance Separation')
res_main_plot = ax1.plot(1000*mean_neg_puc[:, 1], -
mean_neg_level[:, 1], 'rx-', label=r'Level_2$\rightarrow$
Resistance Separation')
res_main_plot = ax1.plot(1000*mean_neg_puc[:, 2], -
mean_neg_level[:, 2], 'bx-', label=r'Level_3$\rightarrow$
Resistance Separation')
res_main_plot = ax1.plot(1000*mean_neg_puc[:, 3], -
mean_neg_level[:, 3], 'gx-', label=r'Level_4$\rightarrow$
Resistance Separation')
ax1.set_xlabel(r'Pulse Current (mA)')
ax1.set_ylabel(r'Difference in Level Resistance$_{xy}$ ($\Omega$)')

```




```

    Omega$))
plt.legend(title=r'Negative_Pulses, Mean_of_Both_Sets,' + '\n
' + f'Mean_Pulse_Separation_of_{np.round(
    full_neg_time_mean_mean, 2)}s')
fig1.set_size_inches(12, 9)
fig1.savefig('puc_nega_mean' + '.pdf')
plt.show()

# plots the pulse separation against the mean positive pulse'
s pulse current
full_pos_time_mean_mean = np.mean(np.array([
    full_pos_time_mean_1, full_pos_time_mean_2]))
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 0], -
    mean_pos_level[:, 0], 'kx-', label=r'Level_1$\rightarrow$
    2_Resistance_Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 1], -
    mean_pos_level[:, 1], 'rx-', label=r'Level_2$\rightarrow$
    3_Resistance_Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 2], -
    mean_pos_level[:, 2], 'bx-', label=r'Level_3$\rightarrow$
    4_Resistance_Separation')
res_main_plot = ax1.plot(1000*mean_pos_puc[:, 3], -
    mean_pos_level[:, 3], 'gx-', label=r'Level_4$\rightarrow$
    5_Resistance_Separation')
ax1.set_xlabel(r'Pulse_Current_(mA)')
ax1.set_ylabel(r'Difference_in_Level_Resistance$__{xy}$$_{($\
    Omega$)})')
plt.legend(title=r'Positive_Pulses, Mean_of_Both_Sets,' + '\n
' + f'Mean_Pulse_Separation_of_{np.round(
    full_pos_time_mean_mean, 2)}s')
fig1.set_size_inches(12, 9)
fig1.savefig('puc_posi_mean' + '.pdf')
plt.show()

```

A.4.3 Multilevel Gap Analysis: Pulse Separation - splitSepAnalysis.py

```

import numpy as np
import time
# import instruments
import winsound
import re
from tkinter import filedialog as dialog
import matplotlib.pyplot as plt
import matplotlib as mpl
import sys
import os
from tqdm import tqdm

```



```

import scipy.optimize as spom

mpl.use("pgf") # sets up mpl to use latex
mpl.rcParams['font.size'] = 18
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "pgf.preamble": [
        r"\usepackage[utf8x]{inputenc}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{cmbright}",
    ]
}
mpl.rcParams.update(style)

file_list = dialog.askopenfilenames(title="Open Data Files")
    # open the data files you want to analyse
pre_list = [f.replace('_bfdh', '_pre_bfdh') for f in
    file_list] # sorts out the names of the other files to
    load
post_list = [f.replace('_bfdh', '_post_bfdh') for f in
    file_list]
prop_list = [f.replace('_bfdh', '_prop_bfdh') for f in
    file_list]

full_pos_level = np.array([], dtype=np.int64).reshape(0, 4)
    # sets up arrays to concatenate into
full_pos_time = np.array([], dtype=np.int64).reshape(0, 4)
full_neg_level = np.array([], dtype=np.int64).reshape(0, 4)
full_neg_time = np.array([], dtype=np.int64).reshape(0, 4)

for i in range(len(file_list)): # loops through all the
    files for all the pulse currents

    data = np.genfromtxt(file_list[i]) # loads up the file
        data
    pre_data = np.genfromtxt(pre_list[i])
    post_data = np.genfromtxt(post_list[i])
    prop_data = np.genfromtxt(prop_list[i])

    data_num = 5 # number of sections of data in that file
    pre_num = 4
    post_num = 4

    pulse_t = data[0:int(len(data)/data_num)] # grabs the
        necessary data from the files

```



```

probe_t = data[int(len(data)/data_num):int(2*len(data)/
data_num)]
vxx = data[int(2*len(data)/data_num):int(3*len(data)/
data_num)]
vxy = data[int(3*len(data)/data_num):int(4*len(data)/
data_num)]
curr = data[int(4*len(data)/data_num):]

pre_pulse_vxx = pre_data[0:int(len(pre_data)/pre_num)] #
    grabs the necessary data from the files
pre_pulse_vxy = pre_data[int(len(pre_data)/pre_num):int
(2*len(pre_data)/pre_num)]
pre_pulse_curr = pre_data[int(2*len(pre_data)/pre_num):
int(3*len(pre_data)/pre_num)]
pre_probe_t = pre_data[int(3*len(pre_data)/pre_num):]

post_pulse_vxx = post_data[0:int(len(post_data)/post_num)
] # grabs the necessary data from the files
post_pulse_vxy = post_data[int(len(post_data)/post_num):
int(2*len(post_data)/post_num)]
post_pulse_curr = post_data[int(2*len(post_data)/post_num
):int(3*len(post_data)/post_num)]
post_probe_t = post_data[int(3*len(post_data)/post_num):]

probe_current = prop_data[0] # grabs the necessary
    property values from the files
pulse_current = prop_data[1]
pulse_loop = int(prop_data[2])
pulse_num = int(prop_data[3])
sleep_time = prop_data[4]
sleep_time2 = prop_data[5]
pulse_width = prop_data[6]
sep_width = prop_data[7]
up_down_width = prop_data[8]
meas_num = int(prop_data[9])
pre_pulse_num = int(prop_data[10])
pre_pulse_pause = prop_data[11]
post_pulse_num = int(prop_data[12])
post_pulse_pause = prop_data[13]

pulse_sep = np.diff(pulse_t) # calculates the time gaps
    between succesive pulses
pulse_sep = pulse_sep[pulse_sep != 0] # gets rid of the
    ones that are zero
rxy = vxy/curr # calculates rxy
rxy_pulse = rxy[1::meas_num] # picks the value of the
    resistance from the second measurement in every pulse

```



```

level_sep = np.diff(rxy_pulse) # calculates the
                                difference

same_time_set = np.zeros((2 * pulse_loop, pulse_num-1))
    # preallocates for arrays
same_level_set = np.zeros((2 * pulse_loop, pulse_num-1))
for i in range(2*pulse_loop):
    same_time_set[i, :] = pulse_sep[i * pulse_num:(i + 1)
        * pulse_num - 1] # calculates the time
        separation
    same_level_set[i, :] = level_sep[i * pulse_num:(i +
        1) * pulse_num - 1] # calculates the level
        separation
pos_level_sep = same_level_set[:, 2, :] # grabs the
    positive level separations
pos_time_sep = same_time_set[:, 2, :]
neg_level_sep = same_level_set[:, 1, :] # grabs the
    negative level separations
neg_time_sep = same_time_set[:, 1, :]

# concatenates arrays together to record the values found
full_pos_level = np.concatenate((full_pos_level,
    pos_level_sep), 0)
full_pos_time = np.concatenate((full_pos_time,
    pos_time_sep), 0)
full_neg_level = np.concatenate((full_neg_level,
    neg_level_sep), 0)
full_neg_time = np.concatenate((full_neg_time,
    neg_time_sep), 0)

mean_neg_level = np.mean(np.array([full_neg_level[:, 2, :],
    full_neg_level[:, 1, :]]), axis=0) # finds the mean value
    for each file/mean pulse separation
mean_neg_time = np.mean(np.array([full_neg_time[:, 2, :],
    full_neg_time[:, 1, :]]), axis=0)
mean_pos_level = np.mean(np.array([full_pos_level[:, 2, :],
    full_pos_level[:, 1, :]]), axis=0)
mean_pos_time = np.mean(np.array([full_pos_time[:, 2, :],
    full_pos_time[:, 1, :]]), axis=0)

# plots the pulse level separation against the 1st negative
    pulse's pulse separation
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(full_neg_time[:, 2, 0],
    full_neg_level[:, 2, 0], 'kx-', label=r'Level_1_\
    rightarrow_2_Resistance_Separation')
res_main_plot = ax1.plot(full_neg_time[:, 2, 1],

```



```

    full_neg_level[:, 2], 'rx-', label=r'Level2$\rightarrow$3ResistanceSeparation')
res_main_plot = ax1.plot(full_neg_time[:, 2],
    full_neg_level[:, 2], 'bx-', label=r'Level3$\rightarrow$4ResistanceSeparation')
res_main_plot = ax1.plot(full_neg_time[:, 3],
    full_neg_level[:, 3], 'gx-', label=r'Level4$\rightarrow$5ResistanceSeparation')
ax1.set_xlabel(r'PulseSeparation(s)')
ax1.set_ylabel(r'DifferenceinLevelResistance$_{xy}$($\Omega$)')
plt.legend(title=r'PositivePulses,1stSet,pulsecurrentof' + '\n' + '18.2mA')
fig1.set_size_inches(12, 6)
fig1.savefig('sep_pos1' + '.pdf')
plt.show()

# plots the pulse level separation against the 2nd negative
# pulse's pulse separation
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(full_neg_time[1::2, 0],
    full_neg_level[1::2, 0], 'kx-', label=r'Level1$\rightarrow$2ResistanceSeparation')
res_main_plot = ax1.plot(full_neg_time[1::2, 1],
    full_neg_level[1::2, 1], 'rx-', label=r'Level2$\rightarrow$3ResistanceSeparation')
res_main_plot = ax1.plot(full_neg_time[1::2, 2],
    full_neg_level[1::2, 2], 'bx-', label=r'Level3$\rightarrow$4ResistanceSeparation')
res_main_plot = ax1.plot(full_neg_time[1::2, 3],
    full_neg_level[1::2, 3], 'gx-', label=r'Level4$\rightarrow$5ResistanceSeparation')
ax1.set_xlabel(r'PulseSeparation(s)')
ax1.set_ylabel(r'DifferenceinLevelResistance$_{xy}$($\Omega$)')
plt.legend(title=r'PositivePulses,2ndSet,pulsecurrentof' + '\n' + '18.2mA')
fig1.set_size_inches(12, 6)
fig1.savefig('sep_pos2' + '.pdf')
plt.show()

# plots the pulse level separation against the 1st positive
# pulse's pulse separation
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(full_pos_time[:, 0],
    full_pos_level[:, 0], 'kx-', label=r'Level1$\rightarrow$2ResistanceSeparation')

```



```

res_main_plot = ax1.plot(full_pos_time[:, 1],
    full_pos_level[:, 1], 'rx-', label=r'Level_2$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(full_pos_time[:, 2],
    full_pos_level[:, 2], 'bx-', label=r'Level_3$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(full_pos_time[:, 3],
    full_pos_level[:, 3], 'gx-', label=r'Level_4$\rightarrow$Resistance Separation')
ax1.set_xlabel(r'Pulse Separation (s)')
ax1.set_ylabel(r'Difference in Level Resistance$_{xy}$ ($\Omega$)')
plt.legend(title=r'Negative Pulses, 1st Set, pulse current of
    ' + '\n' + '18.2 mA')
fig1.set_size_inches(12, 6)
fig1.savefig('sep_neg1' + '.pdf')
plt.show()

```

```

# plots the pulse level separation against the 2nd positive
# pulse's pulse separation
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(full_pos_time[1::2, 0],
    full_pos_level[1::2, 0], 'kx-', label=r'Level_1$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(full_pos_time[1::2, 1],
    full_pos_level[1::2, 1], 'rx-', label=r'Level_2$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(full_pos_time[1::2, 2],
    full_pos_level[1::2, 2], 'bx-', label=r'Level_3$\rightarrow$Resistance Separation')
res_main_plot = ax1.plot(full_pos_time[1::2, 3],
    full_pos_level[1::2, 3], 'gx-', label=r'Level_4$\rightarrow$Resistance Separation')
ax1.set_xlabel(r'Pulse Separation (s)')
ax1.set_ylabel(r'Difference in Level Resistance$_{xy}$ ($\Omega$)')
plt.legend(title=r'Negative Pulses, 2nd Set, pulse current of
    ' + '\n' + '18.2 mA')
fig1.set_size_inches(12, 6)
fig1.savefig('sep_neg2' + '.pdf')
plt.show()

```

```

# plots the pulse level separation against the mean negative
# pulse's pulse separation
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(mean_neg_time[:, 0], -mean_neg_level
   [:, 0], 'kx-', label=r'Level_1$\rightarrow$Resistance Separation')

```



```

    Separation')
res_main_plot = ax1.plot(mean_neg_time[:, 1], -mean_neg_level
   [:, 1], 'rx-', label=r'Level_2$\rightarrow$Resistance_
    Separation')
res_main_plot = ax1.plot(mean_neg_time[:, 2], -mean_neg_level
   [:, 2], 'bx-', label=r'Level_3$\rightarrow$Resistance_
    Separation')
res_main_plot = ax1.plot(mean_neg_time[:, 3], -mean_neg_level
   [:, 3], 'gx-', label=r'Level_4$\rightarrow$Resistance_
    Separation')
ax1.set_xlabel(r'Pulse_Separation(s)')
ax1.set_ylabel(r'Difference_in_Level_Resistance$_{xy}$($\Omega$)')
plt.legend(title=r'Negative_Pulses, Mean_of_Both_Sets, pulse_
    current_of' + '\n' + '18.2$\mu$A')
fig1.set_size_inches(12, 9)
fig1.savefig('sep_nega_mean' + '.pdf')
plt.show()

# plots the pulse level separation against the mean positive
# pulse's pulse separation
fig1, ax1 = plt.subplots(1, 1)
res_main_plot = ax1.plot(mean_pos_time[:, 0], -mean_pos_level
   [:, 0], 'kx-', label=r'Level_1$\rightarrow$Resistance_
    Separation')
res_main_plot = ax1.plot(mean_pos_time[:, 1], -mean_pos_level
   [:, 1], 'rx-', label=r'Level_2$\rightarrow$Resistance_
    Separation')
res_main_plot = ax1.plot(mean_pos_time[:, 2], -mean_pos_level
   [:, 2], 'bx-', label=r'Level_3$\rightarrow$Resistance_
    Separation')
res_main_plot = ax1.plot(mean_pos_time[:, 3], -mean_pos_level
   [:, 3], 'gx-', label=r'Level_4$\rightarrow$Resistance_
    Separation')
ax1.set_xlabel(r'Pulse_Separation(s)')
ax1.set_ylabel(r'Difference_in_Level_Resistance$_{xy}$($\Omega$)')
plt.legend(title=r'Positive_Pulses, Mean_of_Both_Sets, pulse_
    current_of' + '\n' + '18.2$\mu$A')
fig1.set_size_inches(12, 9)
fig1.savefig('sep_posi_mean' + '.pdf')
plt.show()

```

Appendix B Modelling Code

B.1 Simulation - modelSimulation.py



```
import numpy as np
from tqdm import tqdm
import os
import time

def exchange(J, s1, s2): # function that calculates the
    exchange energy contribution between two spins
    spprod = np.cos(s2 - s1) # dot product of the spins
    Eex = -J * spprod # exchange energy calculation
    return Eex

def anis(k, th): # function that calculates the anisotropic
    energy of a spin
    Ek = -k * (np.sin(th)) ** 2 # anisotropic energy
    calculation
    return Ek

def diff(Eb, Ea): # function that calculates difference in
    energy between the
    delE = Eb - Ea # difference calculation
    return delE

def Prob(dE, T): # function that decides whether that
    specific spin can change to it's new position
    kb = 1 # boltzman constant is 1 for this scale
    p = np.exp(-dE / (T * kb)) # uses the boltzmann
    distribution to calculate the probability that the
    spin changes
    test = np.random.random() # generates a random number
    between 0 and 1 to decide whether to turn the spin
    if test < p: # returns a 1 if the spin will turn on that
        condition, 0 if not
        return 1
    else:
        return 0

J = 100 # exchange energy scale factor
k = 2 # anisotropic energy scale factor
B = 10000000 # old and unused, kept for consistency of saved
    data
T = 10 # temperature simulated at
limit = 100 # size of simulation (100x100)
```




```

lol = 500 # number of frames of animation
step = int(np.around((limit ** 2), 0)) # number of spins to
    look at changing in one frame
spread = 0.1 # old and unused, kept for consistency of saved
    data
lop = 50 # old and unused, kept for consistency of saved
    data

anggrid = 2 * np.pi * np.random.random((limit, limit)) - np.
    pi # generates the random grid of angles needed
angtens = np.zeros((limit, limit, lol)) # contains the
    history of the anggrid to save it
delE = np.zeros((lol, step)) # keeps change in energy in
    grid (originally needed for monitoring)
turn = np.zeros((lol, step)) # keeps turn in grid (
    originally needed for monitoring)

for j in tqdm(range(0, lol)): # loop for each frame
    for h in range(0, step): # loop for each change in each
        frame
        locr = np.random.randint(0, limit) # chooses a
            random row for the spin
        locc = np.random.randint(0, limit) # chooses a
            random column for the spin
        exchangelist = np.zeros((4)) # sets up place to
            store exchange energy values from each of the four
            spin exchange interactions
        for i in range(0, 4): # goes through the four
            closest spins to the current spin to calculate the
            exchange energy
            if (i % 2) == 0: # checks if number is even,
                easy way to get it to check both up/down and
                left/right
                exchangelist[i] = exchange(J, anggrid[locr,
                    locc], anggrid[((locr + (i - 1)) % limit),
                    (locc % limit)]) # the two spins go into
                    this to have there exchange energy
                    calculated
            else:
                exchangelist[i] = exchange(J, anggrid[locr,
                    locc], anggrid[(locr % limit), ((locc + (i
                    - 2)) % limit)]) # the two spins go into
                    this to have there exchange energy
                    calculated

        an = anis(k, anggrid[locr, locc]) # calculates the
            anisotropic energy contribution

```



```

Es = an + np.sum(exchangelist) # sums the total
    exchange energy and anisotropic energy
randturn = 2 * np.pi * np.random.random() #
    calculates a random turn for to see if the spin
    wants to turn that much
exchangelist2 = np.zeros((4)) # same as prior
    exchange list
for i in range(0, 4): # loops through neighbouring
    spins with this new angular position
    if (i % 2) == 0:
        exchangelist2[i] = exchange(J, (anggrid[locr,
            locc] + randturn), anggrid[((locr + (i -
                1)) % limit), (locc % limit)])
    else:
        exchangelist2[i] = exchange(J, (anggrid[locr,
            locc] + randturn), anggrid[(locr % limit)
            , ((locc + (i - 2)) % limit)])

an2 = anis(k, (anggrid[locr, locc] + randturn)) #
    calculates the anisotropic energy for this new
    angular position
Ef = an2 + np.sum(exchangelist2) # sums the total
    exchange energy and anisotropic energy for the new
    angular position
delE[j, h] = diff(Es, Ef) # calculates the
    difference in energy between the old and new
    angular positions
turn[j, h] = Prob(delE[j, h], T) # decides whether
    the spin will turn to the new angular position
if turn[j, h] == 1: # performs the turn if previous
    line returns a 1
    anggrid[locr, locc] = anggrid[locr, locc] +
        randturn
else:
    pass

angtens[:, :, j] = anggrid # saves the current position
    of the angular grid

current_datetime = time.strftime("%Y-%m-%d_%H-%M-%S") #
    produces the current date and time (so names don't
    conflict)
np.save(os.getcwd() + '\Data\AFMrunangular' +
    current_datetime, angtens) # saves that angular position
    through time in a file
np.save(os.getcwd() + '\Data\AFMdatetime' + current_datetime,

```



```

        current_datetime) # saves the date and time produced (so
        we can match the names)
needed_data = [J, k, B, T, limit, lol, step, spread, lop] #
    puts the needed parameters together
np.save(os.getcwd() + '\Data\AFMneededdata' +
        current_datetime, needed_data) # saves the needed
        parameters to a file

```

B.2 Graphing - modelAnalysis.py

```

import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
from tkinter import filedialog as dialog
import os
import matplotlib as mpl
from matplotlib import animation

datetime_filename = dialog.askopenfilename(title="Open Date
    and Time file") # creates a dialog to choose the files
    where the date and time data is stored
neededdata_filename = dialog.askopenfilename(title="Open
    Needed Data file") # creates a dialog to choose the files
    where the needed data (properties) is stored
angtens_filename = dialog.askopenfilename(title="Open Angular
    Data file") # creates a dialog to choose the files where
    the angular data is stored

current_datetime = np.load(datetime_filename) # load the
    date and time from the file
current_datetime = str(current_datetime) # change the date
    and time to a string

J, k, B, T, limit, lol, step, spread, lop = np.load(
    neededdata_filename) # Load the needed data (properties)
    from the file
limit = int(limit) # convert limit to integer
lol = int(lol) # convert the frame number to integer
step = int(step) # convert the step number to integer

angtens = np.load(angtens_filename) # load the angular data
    from the file
colang = np.cos(2*angtens) # calculate cos(2*angle) for
    every element

x = np.arange(0, limit) # sets up x range
y = np.arange(0, limit) # sets up y range

```



```

X, Y = np.meshgrid(x, y) # sets up grid of the x and y
    values to plot them

scale = 1 # sets a scale for the arrows
angx = scale * np.cos(angtens) # calculates the u vector
    amount (for when doing smaller scale tests using arrows)
angy = scale * np.sin(angtens) # calculates the v vector
    amount (for when doing smaller scale tests using arrows)

fig, ax = plt.subplots(1, 1) # initial set up for plot
R = ax.pcolormesh(X, Y, colang[:, :, 0], cmap='cividis',
    shading='gouraud', alpha=1) # starts plotting the colour
    contour plot
Q = ax.quiver(X, Y, angx[:, :, 0], angy[:, :, 0], pivot='mid',
    color='k', units='inches', angles='uv', alpha=0) #
    plots the vector plot (when needed)
ax.xaxis.set_ticklabels([]) # gets rid of the tick labels
ax.yaxis.set_ticklabels([]) # gets rid of the tick labels
ax.xaxis.set_ticks([]) # gets rid of the ticks
ax.yaxis.set_ticks([]) # gets rid of the ticks
ax.set_xlim(-1, limit) # sets the x limits
ax.set_ylim(-1, limit) # sets the y limits
fig.set_size_inches(8, 8) # sets the size of the figure
ax.axis('square') # equalises the aspect on each side

pbar = tqdm(total=(lol + 1)) # initialises the progress bar

def update_quiver(num, Q, X, Y): # updates the vector plot
    and colour contour plot
    R.set_array(colang[:, :, num].ravel()) # sets the
        contour plot for that iteration
    U = angx[:, :, num] # gets the U values for the vector
        plot for that iteration
    V = angy[:, :, num] # gets the V values for the vector
        plot for that iteration
    Q.set_UVC(U, V) # sets the vector plot for that
        iteration
    pbar.update(1) # updates the progress bar
    return Q,

anim = animation.FuncAnimation(fig, update_quiver, frames=lol
    , fargs=(Q, X, Y), interval=0.1, blit=False, repeat=False)
    # creates the animation function
video_filename = os.getcwd() + r'\Videos\AFMrun2_size-' + str
    (limit) + '_10xspread-' + str(10*spread) + '_frames-' +
    str(lol) + '_stepchanges-' + str(step) + '_' +

```



```

    current_datetime # sets up the video filename
anim.save(video_filename + '.mp4', fps=60, extra_args=['-
    vcodec', 'libx264'], savefig_kwargs={'pad_inches': 1}) #
    saves the animation function to an mp4

mpl.use("pgf") # sets up mpl and to use latex
mpl.rcParams['font.size'] = 18
mpl.rcParams["savefig.format"] = 'pdf'
style = {
    "pgf.texsystem": "pdflatex",
    "text.usetex": True,
    "pgf.preamble": [
        r"\usepackage[utf8x]{inputenc}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{cmbright}",
    ]
}
mpl.rcParams.update(style)

avgturn = np.mean(colang, axis=(0, 1)) # calculates the mean
    of cos(2*angle).
x = np.arange(0, 15000)/(60*3) # sets up the values for the
    x-axis

fig2, ax2 = plt.subplots(1, 1) # initialises the plot
ax2.plot(x, avgturn, 'k-') # plots the data
ax2.set_xlabel('Time into Animation (s)') # labels the x-
    axis
ax2.set_ylabel(r'Mean Value of  $\cos(2 \cdot \theta)$ 
    right)$') # labels the y-axis
fig2.set_size_inches(12, 9) # sets the size of the figure
graph_filename = os.getcwd() + r'\Graphs\AFMrun2_size-' + f'{
    limit}_10xspread-{10*spread}_frames-{lol}_stepchanges-{
    step}' + current_datetime # sets up the graph filename
fig2.savefig(graph_filename + '.pdf') # saves the graph

```

Appendix C Notes

The Electronic Appendices will accompany this report, they contain the modelling videos, every piece of code shown in the appendices, and all of the figures in this report for easier viewing.

